Chapter 22 — Interview Question Bank

> **What this chapter is.** Python interview questions, worded the way interviewers and the popular public lists word them, each with a short answer you can say out loud and a link to the place in this book that explains it in full. It collects the questions that come up again and again in interview-experience posts, the big GitHub question collections, widely shared prep lists and data-team interview guides. Chapter 20 already answers about 45 core questions; this bank links to those answers and adds every common question Chapter 20 does not cover, including "what does this print?" puzzles, the small coding tasks Python rounds use, and NumPy/pandas basics. Versions: Python 3.14 (answers also hold for 3.12+ unless a version is named), numpy 2.x, pandas 3.0.

How to use this bank

- **Wording.** Each question is phrased the way it is usually asked. Several phrasings of one idea are merged into one question.
- **★ = asked very often.** The question appears in many of the sources listed at the end. If you have little time, drill only the ★ questions first (there are 100).
- **Answers.** Each answer is 2–6 sentences, or a short code snippet plus a sentence or two: what you would actually say in an interview. Terms are defined the first time they appear.
- **→ More.** Points to the chapter and section that explains the topic fully. "Ch 20 Q*n*" means question *n* of the Q&A in Chapter 20 (§20.3), and "gotcha *n*" means gotcha *n* in §20.2. A question with no "More" line is answered only here.
- **How to drill.** Cover the answer, say yours out loud in under a minute, then check. Mark the ones you missed and repeat them the next day. For the puzzles (Section 13), write down the output *before* reading the answer. For the coding tasks (Section 14), type the solution from memory and run it.

Contents: 1. Basics and how Python runs · 2. Data types, mutability and strings · 3. Collections · 4. Functions, arguments, scope and closures · 5. OOP, dunder methods and the MRO · 6. Iterators, generators and comprehensions · 7. Decorators, context managers and metaprogramming · 8. Exceptions and files · 9. Modules, packaging, environments and type hints · 10. Memory model, internals and performance · 11. Concurrency: GIL, free-threading, threads, processes, asyncio · 12. Standard library, testing and debugging · 13. "What does this print?" puzzles · 14. Common coding tasks in Python rounds · 15. NumPy and pandas basics.

---

1. Basics and how Python runs

Q1. ★ What is Python, and what are its key features? Python is a high-level (it hides memory management and hardware details), general-purpose programming language created by Guido van Rossum and first released in 1991. Key features: readable syntax that uses indentation for blocks; dynamic, strong typing; automatic memory management; multiple paradigms (procedural, object-oriented, functional); a large standard library ("batteries included") plus the PyPI package index; and easy integration with C/C++/Rust code. It is used for web back ends, automation and scripting, data analysis, machine learning and testing. Its main costs are slower raw execution than compiled languages and limited thread parallelism in the default build (the GIL, Q197). → *More:* Ch 1 §1.2–1.3 — What Python is, where it is used

Q2. ★ Is Python compiled or interpreted? How is Python code executed? Both, in stages. CPython (the standard implementation, written in C) parses your source into an AST (abstract syntax tree, a tree form of the code), compiles that into **bytecode** (instructions for Python's own virtual machine), then its interpreter loop executes the bytecode. Nothing is compiled ahead of time to machine code by default; an experimental JIT (just-in-time compiler) exists since 3.13 but is off unless you enable it. You can see the bytecode with `dis.dis(func)`. → *More:* Ch 20 Q1 · Ch 1 §1.8 — How CPython runs your code

Q3. What is a `.pyc` file and `__pycache__`? Why does Python create `.pyc` files if it is interpreted? A `.pyc` file is the cached bytecode of an imported module, stored in a `__pycache__` folder and named with the interpreter version (for example `utils.cpython-314.pyc`). On the next import, Python checks whether the source changed (by timestamp or hash) and, if not, loads the bytecode directly and skips the compile step, so start-up is faster. The script you run directly (`python app.py`) is compiled every time and not cached. Deleting `__pycache__` is always safe. → *More:* Ch 1 §1.8 · Ch 8 §8.2 — How `import` really works

Q4. What are CPython, PyPy, Jython and MicroPython? When would you use an alternative implementation? Python is a language specification; an *implementation* is a program that runs it. **CPython** is the reference implementation and what "Python" usually means. **PyPy** has a tracing JIT and is often several times faster on long-running pure-Python loops, but C-extension support is weaker. **MicroPython** runs on microcontrollers. **Jython** (JVM) and **IronPython** (.NET) exist but lag behind modern Python 3. Pick an alternative only for a specific need (speed of pure-Python code, embedded hardware); most libraries target CPython. → *More:* Ch 1 §1.2 · Ch 17 §17.7

Q5. ⋆ What is PEP 8? What is PEP 20? A PEP (Python Enhancement Proposal) is a design document for the language. **PEP 8** is the style guide: 4-space indentation, `snake_case` for functions and variables, `PascalCase` for classes, `UPPER_CASE` for constants, lines of about 79 characters (many teams use 88–100), imports grouped at the top, two blank lines between top-level definitions. **PEP 20**, "The Zen of Python" (`import this`), lists the design values, such as "Readability counts" and "Explicit is better than implicit". In practice, formatters and linters (black, ruff) enforce PEP 8 automatically. → *More:* Ch 1 §1.11 — PEP 8 and PEP 20

Q6. ⋆ What does "dynamically typed" and "strongly typed" mean? What is duck typing? *Dynamic*: a variable has no fixed type; the object it refers to has one, and types are checked when the code runs. *Strong*: Python does not silently convert between unrelated types, so `"1" + 1` raises `TypeError`. **Duck typing** ("if it walks like a duck and quacks like a duck…") means code relies on what an object can *do* rather than what class it is: any object with a `read()` method can be passed where a file is expected. → *More:* Ch 20 Q2 · Ch 2 §2.3 · Ch 10 §10.7

Q7. ⋆ What does "everything in Python is an object" mean? Every value — numbers, strings, functions, classes, modules, even `None` — is an object with an identity (`id()`), a type (`type()`) and a value. So functions can be stored in lists and passed as arguments, classes can be created and modified at runtime, and you can call methods on literals (`(5).bit_length()`). Variables are just names that refer to objects. → *More:* Ch 17 §17.1 — The CPython object model · Ch 2 §2.1

Q8. Why is indentation important in Python? Is Python case-sensitive? Indentation is syntax: it defines blocks (the body of `if`, `for`, `def`, `class`) instead of braces, so wrong indentation causes `IndentationError` or, worse, a different program. Use 4 spaces and never mix tabs and spaces. Python is case-sensitive: `count`, `Count` and `COUNT` are three different names. → *More:* Ch 1 §1.9 — Basic syntax

Q9. What is the difference between an expression and a statement? An **expression** produces a value (`2 + 3`, `f(x)`, `a if c else b`, a lambda body). A **statement** performs an action and has no value (`if`, `for`, `def`, `import`, `return`, assignment). That is why a `lambda` can contain only an expression. The walrus operator `:=` is an *assignment expression*: it assigns and yields the value, so it can appear inside an expression. → *More:* Ch 1 §1.9 · Ch 2 §2.8

Q10. What is the difference between `pass`, `continue` and `break`? `pass` does nothing; it is a placeholder where a statement is required (an empty function or class). `continue` skips the rest of the current loop iteration and goes to the next one. `break` leaves the innermost loop immediately (and skips the loop's `else` clause). → *More:* Ch 3 §3.8 — Loop control

Q11. What does `else` on a `for` or `while` loop do? The `else` block runs only when the loop finishes normally, that is, without hitting `break`. It is the clean way to write "search, and if not found do X" without a flag variable:

```
for n in numbers:
    if n < 0:
        print("found a negative:", n)
        break
else:
    print("no negatives")
```

→ *More:* Ch 3 §3.9 — for-else and while-else

Q12. What is the walrus operator `:=`? An assignment expression (Python 3.8+, PEP 572) that assigns a value to a name *and* returns it, so you can test and keep a value in one step: `while (line := f.readline()):` or `if (m := pattern.search(s)):`. It also avoids computing something twice in a comprehension: `[y for x in data if (y := f(x)) > 0]`. Use it where it removes duplication, not everywhere. → *More:* Ch 2 §2.8 · Ch 3

Q13. What is `match`/`case`? How is it different from a `switch` statement? Structural pattern matching (3.10+) compares a value against *patterns*, not just constants. It can destructure sequences (`case [x, y]:`), mappings (`case {"type": "click", "pos": (x, y)}:`), class instances (`case Point(x=0):`), combine alternatives (`case 401 | 403:`) and add guards (`case [x, *_] if x > 0:`). There is no fall-through, and `case _:` is the default. A bare name in a pattern *captures* (binds) the value rather than comparing with a variable; use dotted names (`Color.RED`) for constants. → *More:* Ch 3 §3.4 — match/case

Q14. What is a ternary (conditional) expression in Python? `value_if_true if condition else value_if_false`, for example `label = "even" if n % 2 == 0 else "odd"`. It is an expression, so it can be used inside function calls and comprehensions. Avoid nesting more than one. → *More:* Ch 3 §3.3

Q15. What is a docstring? How is it different from a comment? A docstring is a string literal written as the first statement of a module, class or function. Python stores it in `__doc__`, and `help()`, IDEs and documentation generators read it. A comment (`#`) is thrown away by the compiler and is for readers of the source only. Docstrings say *what* the code does and how to use it; comments explain *why* something non-obvious is done. → *More:* Ch 1 §1.10 · Ch 7 §7.8

Q16. What do `help()`, `dir()`, `type()` and `id()` do? `help(obj)` shows its documentation. `dir(obj)` lists its attribute names. `type(obj)` returns its class. `id(obj)` returns a number unique to the object while it is alive (in CPython, its memory address). Together with `vars()`, `getattr()` and the `inspect` module they are Python's **introspection** tools (examining objects at runtime). → *More:* Ch 12 §12.8 — Introspection

Q17. What is new in recent Python versions (3.12, 3.13, 3.14)? **3.12**: type-parameter syntax `def first[T](xs: list[T]) -> T` and the `type` alias statement; f-strings can reuse the same quotes and nest freely; better error messages. **3.13**: a new interactive REPL; an optional free-threaded (no-GIL) build and an experimental JIT, both off by default. **3.14**: the free-threaded build becomes officially supported but is still optional (PEP 779); annotations are evaluated lazily (PEP 649/749, new `annotationlib`); template strings `t"..."` (PEP 750); `except A, B:` without parentheses (PEP 758); a `SyntaxWarning` for `return`/`break`/`continue` leaving a `finally` block (PEP 765); multiple interpreters in the standard library (`concurrent.interpreters`, PEP 734); Zstandard compression (`compression.zstd`); `forkserver` as the default multiprocessing start method on Linux. → *More:* Ch 20 Q9 · Ch 16 §16.6–16.7 · Ch 13 §13.2

Q18. What are the main differences between Python 2 and Python 3? Still asked, though Python 2 reached end of life in January 2020. In Python 3: `print` is a function; `str` is Unicode text and `bytes` is separate; `/` is true division (`3 / 2 == 1.5`) and `//` is floor division; `range`, `map`, `zip` and `dict.keys()` return lazy objects or views instead of lists; there is a single `int` type with unlimited size; and exception syntax is `except E as e`. The answer interviewers want: "Python 2 is dead; all new code is Python 3."

Q19. What are Python's main limitations? When would you not choose Python? Pure-Python code is slow for CPU-heavy loops (often 10–100× slower than C, Java or Go); the GIL limits thread parallelism in the default build; memory use per object is high; dynamic typing lets some bugs reach runtime; and it is a poor fit for mobile apps, browsers and hard real-time systems. Mitigations: NumPy and other C-backed libraries, multiprocessing, type checkers, and rewriting hot spots in C, Cython or Rust. → *More:* Ch 17 §17.7

---

2. Data types, mutability and strings

Q20. What are the built-in data types in Python? Numbers: `int` (unlimited size), `float` (64-bit IEEE 754 double), `complex`; `bool` (a subclass of `int`); text: `str`; binary: `bytes`, `bytearray`, `memoryview`; sequences: `list`, `tuple`, `range`; mappings: `dict`; sets: `set`, `frozenset`; and `NoneType` (whose only

value is `None`). Immutable ones: `int`, `float`, `complex`, `bool`, `str`, `bytes`, `tuple`, `frozenset`, `range`. → *More:* Ch 2 §2.4 — Built-in data types

Q21. ⋆ What is the difference between mutable and immutable objects? A mutable object can be changed in place after it is created (`list`, `dict`, `set`, `bytearray`, most user classes). An immutable one cannot (`int`, `str`, `tuple`, `frozenset`); an operation that seems to change it builds a new object and rebinds the name. Consequences: only immutable (hashable) values can be dict keys or set members; mutable objects shared between names change for everyone; and immutable objects are safe to share between threads. → *More:* Ch 20 Q3 · Ch 2 §2.2

Q22. What happens when you write `b = a`? Are variables boxes? No copy is made. A variable is a *name* bound to an object; `b = a` binds a second name to the *same* object. If that object is mutable, a change through `b` is visible through `a`. To get an independent object, copy it explicitly (`a.copy()`, `list(a)`, `copy.deepcopy(a)`).

```
a = [1, 2]
b = a
b.append(3)
print(a)        # [1, 2, 3]
```

→ *More:* Ch 2 §2.1 · Ch 5 §5.8

Q23. ⋆ What is the difference between `is` and `==`? `==` compares *values* by calling `__eq__`. `is` compares *identity*: whether both names refer to the same object. Use `is` only for singletons such as `None` (`if x is None`), and `==` for everything else. `[] == []` is `True` but `[] is []` is `False`. → *More:* Ch 20 Q5 and gotchas 3–4 · Ch 2 §2.2

Q24. What is `None`? How is it different from `0`, `False` and `""`? `None` is the single instance of `NoneType` and means "no value" (a function without `return` returns it). `0`, `False` and `""` are real values of other types that happen to be *falsy*. All four are falsy, but only `None is None`. When 0 or an empty string is a valid value, test `if x is None`, not `if not x`. → *More:* Ch 2 §2.6

Q25. Which values are falsy in Python? `None`, `False`, numeric zeros (`0`, `0.0`, `0j`, `Decimal(0)`), and empty containers (`""`, `[]`, `()`, `{}`, `set()`, `range(0)`). Everything else is truthy by default. A class controls its truthiness with `__bool__`, or with `__len__` (zero length means falsy). → *More:* Ch 2 §2.6 — Booleans and truthiness

Q26. How big can an `int` be? How are floats stored? `int` has arbitrary precision: it grows as needed, so `2 ** 1000` is exact (but arithmetic on huge numbers gets slower; and since 3.11, converting an int with more than 4300 digits to `str` raises `ValueError` by default, a guard against denial-of-service attacks, adjustable with `sys.set_int_max_str_digits`). `float` is a 64-bit IEEE 754 binary double with about 15–17 significant decimal digits, so many decimal fractions cannot be stored exactly. `sys.float_info` shows the limits. → *More:* Ch 2 §2.5 — Numbers

Q27. ⋆ Why is `0.1 + 0.2 == 0.3` false? How do you compare or handle money? `0.1` and `0.2` have no exact binary representation, so their sum is `0.30000000000000004`. Compare floats with a tolerance: `math.isclose(a, b)`. For money use `decimal.Decimal` built from strings (`Decimal("0.10")`) or store integer cents. For exact rationals use `fractions.Fraction`. → *More:* Ch 20 gotcha 10 · Ch 14 §14.8

Q28. What is the difference between `/`, `//` and `%`? What about negative numbers? `/` is true division and always returns a `float` (`7 / 2 == 3.5`, `4 / 2 == 2.0`). `//` is floor division: it rounds *down* toward minus infinity (`7 // 2 == 3`, `-7 // 2 == -4`). `%` is the remainder, and its sign follows the divisor, so that `a == (a // b) * b + a % b` always holds (`-7 % 2 == 1`). `divmod(a, b)` returns both. `int(-3.5)` truncates toward zero and gives `-3`. → *More:* Ch 2 §2.8 — Operators

Q29. How does `round()` work? Why is `round(2.5) == 2`? `round` uses "round half to even" (banker's rounding): exact halves go to the nearest even number, so `round(2.5) == 2` and `round(3.5) == 4`. This avoids a bias when rounding many values. Also, `round(2.675, 2)` gives `2.67` because 2.675 is stored as slightly less than that. Use `Decimal.quantize` with `ROUND_HALF_UP` when a business rule demands "school" rounding. → *More:* Ch 2 §2.5

Q30. What do `and` and `or` return? What is short-circuit evaluation? They return one of their *operands*, not necessarily a `bool`. `a or b` returns `a` if it is truthy, otherwise `b`; `a and b` returns `a` if it is falsy, otherwise `b`. Evaluation stops as soon as the result is known (short-circuit), so `x and x.name`

is safe when `x` is `None`. Idiom: `name = user_input or "anonymous"` (careful: it also replaces `0` and `""`). → *More:* Ch 2 §2.8

Q31. How do chained comparisons like `a < b < c` work? `a < b < c` means `a < b and b < c`, with `b` evaluated only once. This works for any comparison operators, which produces surprises such as `1 == 1.0 == True` being `True`, and `x < y > z` being legal. → *More:* Ch 2 §2.8

Q32. How do you convert between types? What is implicit vs explicit conversion? Explicit conversion uses constructors: `int("42")`, `float("3.5")`, `str(10)`, `list("abc")`, `int("ff", 16)`, `int(3.9) == 3`. `int("3.5")` raises `ValueError` (go through `float` first). Implicit conversion happens only between numeric types (`1 + 2.0` gives `3.0`, `True + 1` gives `2`); Python never converts strings to numbers automatically. → *More:* Ch 2 §2.7 — Type conversion

Q33. ⋆ Are strings mutable? How do you "change" a character in a string? No. `s[0] = "x"` raises `TypeError`. Build a new string: `s = "x" + s[1:]`, `s.replace(old, new)`, or convert to a list, edit, and `"".join(...)` it back. Immutability makes strings hashable (usable as dict keys) and safe to share. → *More:* Ch 4 §4.2 — Immutability

Q34. ⋆ What is the best way to concatenate many strings? `"".join(parts)`, which computes the final size once and copies each piece once (linear time). Repeated `s += piece` in a loop may copy the growing string each time (quadratic in the worst case; CPython sometimes optimises it in place, but don't rely on that). For incremental building, append pieces to a list and join once at the end, or use `io.StringIO`. → *More:* Ch 4 §4.8 — Joining strings efficiently

Q35. ⋆ What are the ways to format strings? What are f-strings, and what are t-strings in 3.14? Three classic ways: f-strings `f"{name} is {age}"` (3.6+, fastest and most readable; support format specs such as `{x:.2f}`, `{n:,}`, `{s:>10}` and the debug form `{x=}`), `str.format()`, and the old `%` operator (still used by `logging`). **t-strings** (`t"..."`, 3.14, PEP 750) look like f-strings but produce a `string.templatelib.Template` object holding the literal parts and the interpolated values separately, so a library can escape or process the values (for example to build safe HTML or SQL) before producing the final text. → *More:* Ch 4 §4.6 — String formatting

Q36. ⋆ What is slicing? How do you reverse a string or list? `seq[start:stop:step]` returns a new sequence from `start` up to but not including `stop`; any part can be omitted, and negative indexes count from the end (`s[-1]` is the last item). `s[::-1]` reverses. Slicing never raises `IndexError` for out-of-range bounds (`[1, 2][5:]` is `[]`). On lists, slicing creates a shallow copy; on NumPy arrays, it creates a *view* (Q290). → *More:* Ch 4 §4.4 — Slicing · Ch 5 §5.5

Q37. What is the difference between `str` and `bytes`? What are encoding and decoding? `str` is a sequence of Unicode code points (abstract characters); `bytes` is a sequence of integers 0–255 (raw data). **Encoding** turns text into bytes (`"é".encode("utf-8") == b"\xc3\xa9"`), **decoding** turns bytes back into text. Files, sockets and hashes work with bytes. Always name the encoding explicitly (`open(path, encoding="utf-8")`); a wrong guess causes `UnicodeDecodeError` or garbled text. → *More:* Ch 4 §4.9 — Unicode vs bytes

Q38. What is a raw string (`r"..."`)? A raw string treats backslashes as ordinary characters, so `r"\n"` is two characters, not a newline. Use raw strings for regular expressions (`r"\d+"`) and Windows paths. A raw string cannot end with a single backslash. → *More:* Ch 4 §4.5 — Escape sequences and raw strings

Q39. What is the difference between `find()` and `index()`? Between `split()`, `rsplit()` and `partition()`? `find` returns `-1` when the substring is missing; `index` raises `ValueError`. `split(sep)` returns a list of all pieces (with no argument it splits on runs of whitespace and drops empty strings); `rsplit(sep, maxsplit)` splits from the right; `partition(sep)` always returns a 3-tuple `(before, sep, after)`, which is handy for "key=value" parsing. → *More:* Ch 4 §4.7 — String methods

Q40. What is the difference between `strip("abc")` and `removeprefix("abc")`? `strip`, `lstrip` and `rstrip` take a *set of characters* and remove any of them from the ends: `"cabbage".lstrip("abc")` gives `"ge"`. `removeprefix` and `removesuffix` (3.9+) remove an exact substring once: `"cabbage".removeprefix("cab")` gives `"bage"`. Using `lstrip` to remove a prefix is a classic bug. → *More:* Ch 4 §4.7

Q41. What is the difference between `isdigit()`, `isdecimal()`, `isnumeric()` and `isalnum()`? All test every character and return `False` for an empty string. `isdecimal` is the strictest: only characters usable to write base-10 numbers (`"123"`). `isdigit` also accepts digits like superscript `"²"`. `isnumeric` also accepts numeric characters such as `"½"` and Chinese numerals. `isalnum` is true if each character is a

letter or numeric. To check that a string parses as an int, the robust way is `try: int(s)`. → *More:* Ch 4 §4.7

Q42. What is the difference between `re.match`, `re.search`, `re.fullmatch` and `re.findall`? `match` anchors at the start of the string; `search` finds the first match anywhere; `fullmatch` requires the whole string to match; `findall` returns all non-overlapping matches as strings (or tuples of groups), and `finditer` yields match objects lazily. `re.sub` replaces, `re.split` splits. Compile patterns you reuse with `re.compile`, and use raw strings. → *More:* Ch 4 §4.10 · Ch 14 §14.3 — re, deeper

Q43. What is the difference between greedy and non-greedy matching in regex? Quantifiers `*`, `+`, `?` and `{m,n}` are greedy: they match as much as possible and then back off. Adding `?` (`*?`, `+?`) makes them lazy: as little as possible. On `"<a><b>"`, `<.*>` matches the whole string, while `<.*?>` matches `"<a>"`. → *More:* Ch 14 §14.3

Q44. What is string interning? Interning keeps one shared copy of certain strings so that identical strings are the same object, making dict lookups by those strings faster (an identity check succeeds before comparing characters). CPython interns identifiers and many compile-time constant strings automatically; `sys.intern(s)` does it explicitly. Never rely on it for correctness: compare strings with `==`. → *More:* Ch 17 §17.3 — Small-int caching and string interning · Ch 20 gotcha 4

---

3. Collections: lists, tuples, dicts, sets and `collections`

Q45. ⋆ What is the difference between a list and a tuple? When do you use each? A list is mutable and is used for a variable-length collection of similar items; a tuple is immutable and is used for a fixed-size record whose positions have meaning (`(lat, lon)`, a function returning several values). Tuples are hashable when their items are (so they can be dict keys), slightly smaller, faster to create, and signal "this will not change". → *More:* Ch 20 Q10 · Ch 5 §5.9

Q46. ⋆ How is a Python list implemented? What is the time complexity of its operations? A dynamic array of pointers to objects that over-allocates spare capacity. Index read/write, `len`, `append` and `pop()` from the end are O(1) (append is *amortised* O(1): occasionally the array grows and copies, but the average cost stays constant). `insert(0, x)`, `pop(0)`, `remove(x)`, `x in lst` and `index(x)` are O(n). Sorting is O(n log n). → *More:* Ch 20 Q11 · Ch 5 §5.3 — List internals and big-O

Q47. What is the difference between `append()`, `extend()` and `insert()`? Between `+` and `+=` on lists? `append(x)` adds `x` as one item; `extend(iterable)` adds each item of the iterable; `insert(i, x)` puts `x` before index `i` (O(n)). `a + b` builds a new list; `a += b` extends `a` *in place* (it calls `__iadd__`), so other names bound to the same list see the change. `a += "hi"` extends with `'h', 'i'`, while `a = a + "hi"` raises `TypeError`. → *More:* Ch 5 §5.4 — Adding and removing items

Q48. What is the difference between `del`, `remove()`, `pop()` and `clear()`? `del lst[i]` (or a slice) deletes by position and returns nothing; `lst.remove(x)` deletes the first item equal to `x` and raises `ValueError` if absent; `lst.pop(i=-1)` deletes by position and *returns* the item; `lst.clear()` empties the list in place. `del name` unbinds the name itself, not the object. → *More:* Ch 5 §5.4

Q49. ⋆ What is the difference between `sort()` and `sorted()`? How do you sort by multiple keys or in descending order? `lst.sort()` sorts the list in place and returns `None`; `sorted(iterable)` returns a new list and accepts any iterable. Both use Timsort, are O(n log n) and **stable** (equal items keep their order). Use `key=` to sort by a computed value and `reverse=True` for descending. For several keys, return a tuple: `sorted(people, key=lambda p: (-p.age, p.name))` sorts by age descending, then name ascending. Stability also lets you sort in two passes, least significant key first. → *More:* Ch 5 §5.7 — Sorting · Ch 19 §19.6

Q50. What is unpacking? What does `first, *rest = items` do? Unpacking assigns the items of any iterable to several names at once: `a, b = b, a` swaps; `x, y = point`. A starred name collects the leftovers into a list: `first, *rest = [1, 2, 3]` gives `first == 1`, `rest == [2, 3]`; `*_, last = items` takes the last item. The count must match, or you get `ValueError`. `*` and `**` also unpack inside literals: `[*a, *b]`, `{**d1, **d2}`. → *More:* Ch 5 §5.10 — Unpacking

Q51. ⋆ How does a dictionary work internally? What is its time complexity? A dict is a hash table: `hash(key)` chooses a slot, and collisions are resolved by open addressing (probing other slots). CPython uses a compact layout, an index table pointing into a dense array of entries, which saves

memory and keeps insertion order. Get, set, delete and `in` are O(1) on average and O(n) in the worst case (many collisions). The table is resized when it gets about two-thirds full. → *More:* Ch 20 Q12 · Ch 6 §6.2 — Hashing

Q52. ⋆ What can be used as a dictionary key? Why can't a list be a key? Any **hashable** object: one whose `__hash__` never changes during its lifetime and is consistent with `__eq__` (equal objects have equal hashes). Built-in immutables qualify (`int`, `str`, `tuple` of hashables, `frozenset`). A list is mutable: if its contents changed after insertion, its hash would change and the dict could never find it again, so lists define `__hash__ = None`. Note that `1`, `1.0` and `True` are equal and hash the same, so they are the *same* key. → *More:* Ch 20 Q13 · Ch 6 §6.2

Q53. Are dictionaries ordered? Yes. Since Python 3.7 the language guarantees that dicts keep insertion order (CPython 3.6 already did as an implementation detail). Updating an existing key keeps its position; deleting and re-inserting moves it to the end. `OrderedDict` is still useful for `move_to_end`, `popitem(last=False)` and order-sensitive equality. → *More:* Ch 6 §6.1 · Ch 6 §6.9

Q54. ⋆ How do you merge two dictionaries? `merged = d1 | d2` (3.9+) returns a new dict; `d1 |= d2` or `d1.update(d2)` updates in place; `{**d1, **d2}` works on older versions. For duplicate keys the right-hand value wins. These are shallow merges: nested dicts are not merged recursively (write a small recursive function for that, Q270). → *More:* Ch 6 §6.4 — Dictionary methods

Q55. What is the difference between `d[key]`, `d.get(key)`, `d.setdefault(key, default)` and `defaultdict`? `d[key]` raises `KeyError` if the key is missing. `d.get(key, default)` returns a default without changing the dict. `d.setdefault(key, default)` returns the existing value or inserts and returns the default. `collections.defaultdict(list)` calls the factory automatically on a missing key, ideal for grouping: `groups[k].append(v)`. A dict subclass can also define `__missing__` to decide what happens on a missing key (that is how `defaultdict` works). → *More:* Ch 6 §6.4 · Ch 6 §6.9 · Ch 20 gotcha 7

Q56. What are dictionary views (`keys()`, `values()`, `items()`)? They are live, lazy views of the dict, not copies: they reflect later changes and take no extra memory. `keys()` and `items()` support set operations (`d1.keys() & d2.keys()` gives the common keys). Changing the dict's size while iterating over a view raises `RuntimeError`; iterate over `list(d)` if you need to delete. → *More:* Ch 6 §6.4 — view objects

Q57. How do you invert a dictionary or create one from two lists? From two lists: `dict(zip(keys, values))` (add `strict=True` to fail on unequal lengths). Inverting: `{v: k for k, v in d.items()}`; if values repeat, later keys overwrite earlier ones, so group instead: `inv = defaultdict(list); for k, v in d.items(): inv[v].append(k)`. To give many keys the same value: `dict.fromkeys(keys, 0)` (never with a mutable default, see gotcha 7). → *More:* Ch 6 §6.5 — Dict and set comprehensions

Q58. ⋆ What is a set? Why is `in` faster on a set than on a list? A set is an unordered collection of unique hashable items, implemented as a hash table (a dict with keys only). `x in s` hashes `x` and checks one slot (O(1) on average), while `x in lst` scans every item (O(n)). Sets also remove duplicates (`set(items)`) and support algebra: union `|`, intersection `&`, difference `-`, symmetric difference `^`, and subset `<=`. → *More:* Ch 20 Q14 · Ch 6 §6.6–6.7

Q59. What is a `frozenset`? Can a set contain a list or another set? A `frozenset` is an immutable, hashable set, so it can be a dict key or an element of another set. A set cannot contain a list or a normal set (both are unhashable, `TypeError: unhashable type`), but it can contain tuples and frozensets. Note that `{}` is an empty *dict*; an empty set is `set()`. → *More:* Ch 6 §6.8 — frozenset

Q60. ⋆ What is the difference between a shallow copy and a deep copy? What are the ways to copy a list? A shallow copy creates a new outer container but shares the inner objects; a deep copy recursively copies everything. Shallow: `lst[:]`, `lst.copy()`, `list(lst)`, `copy.copy(lst)`. Deep: `copy.deepcopy(lst)`, which also handles cycles. With nested lists, changing an inner list through a shallow copy changes the original too. → *More:* Ch 20 Q15 and gotcha 6 · Ch 5 §5.8 — Copying

Q61. ⋆ What is in the `collections` module? `Counter` (counts hashable items, `most_common(k)`), `defaultdict` (default values for missing keys), `deque` (double-ended queue), `namedtuple` (tuple with named fields), `OrderedDict` (order-aware dict with `move_to_end`), `ChainMap` (searches several dicts as one, e.g. defaults under overrides) and `UserDict` / `UserList` / `UserString` (base classes that are easier to subclass than the built-ins). → *More:* Ch 20 Q17 · Ch 6 §6.9 — The collections module

Q62. Why use `deque` instead of a list for a queue? `list.pop(0)` and `insert(0, x)` shift every element, O(n). `collections.deque` is a doubly linked list of fixed-size blocks, so `append`, `appendleft`, `pop` and `popleft` are all O(1). `deque(maxlen=n)` keeps only the last *n* items (a sliding window or recent history). Indexing into the middle of a deque is O(n), so use it for ends, not random access. → *More:* Ch 6 §6.9 · Ch 19 §19.4 — Stacks and queues

Q63. How do you use `heapq`? How do you get a max-heap? `heapq` keeps a plain list as a binary min-heap: `heappush(h, x)` and `heappop(h)` are O(log n), `h[0]` is the smallest item, and `heapify(lst)` is O(n). For a max-heap, push negated numbers (`-x`) or tuples with negated priorities (Python 3.14 also adds public max-heap functions such as `heapq.heappush_max`, but the negation trick is what interviewers expect). `heapq.nlargest(k, xs)` and `nsmallest` find the top *k* in O(n log k). Push `(priority, counter, item)` tuples so that ties never compare the items themselves. → *More:* Ch 6 §6.10 — heapq · Ch 19 §19.5 — Heaps

Q64. What does the `bisect` module do? It does binary search on a sorted list: `bisect_left(a, x)` returns the leftmost position where `x` could be inserted to keep order, `bisect_right` the rightmost; `insort(a, x)` inserts in order (O(log n) to find the spot, but O(n) to shift elements). Since 3.10 they accept `key=`. Typical uses: grade boundaries, counting items in a range, maintaining a sorted list. → *More:* Ch 6 §6.11 — bisect · Ch 19 §19.6

Q65. What is a `namedtuple`? How does it compare with a dataclass, a dict and `TypedDict`? `namedtuple("Point", "x y")` (or `typing.NamedTuple`) creates a tuple subclass with named fields: immutable, lightweight, unpackable and indexable. A `@dataclass` is a normal class with generated methods: mutable by default, supports defaults, methods and `frozen=True`. A dict is flexible but has no fixed schema. `TypedDict` only describes the expected keys of a dict for type checkers; at runtime it is a plain dict. Use namedtuple for small immutable records, dataclass for domain objects, dicts for JSON-like data. → *More:* Ch 5 §5.13 — namedtuple · Ch 10 §10.10 · Ch 13 §13.9

Q66. What is the difference between `array.array`, `list` and a NumPy array? A `list` holds pointers to any objects. `array.array("d")` stores raw C numbers of one type compactly but has no vectorised maths. A NumPy `ndarray` stores a typed contiguous block *and* provides fast vectorised operations, broadcasting and many dimensions. Use lists for general data, NumPy for numeric work. → *More:* Ch 20 Q16 · Ch 5 §5.14

Q67. What is the time complexity of common operations on built-in types?

Operation	list	dict / set	deque
index `x[i]`	O(1)	—	O(n)
`x in c`	O(n)	O(1) avg	O(n)
add at end	O(1) amortised	O(1) avg	O(1)
add/remove at front	O(n)	—	O(1)
delete by key/value	O(n)	O(1) avg	O(n)
sort	O(n log n)	—	—
Also: `len()` is O(1) for all; `min` / `max` / `sum` are O(n); slicing `lst[a:b]` is O(b−a); string concatenation in a loop is O(n²) in the worst case.			
→ *More:* Ch 17 §17.5 — Big-O of built-in operations · Ch 19 §19.1			

Q68. Which data structure would you choose for: unique items, counting, a FIFO queue, a priority queue, an LRU cache, fast lookups by key? Unique items → `set`; counting → `Counter`; FIFO queue → `deque` (or `queue.Queue` between threads); priority queue → `heapq` (or `queue.PriorityQueue` between threads); LRU cache → `functools.lru_cache` for function results, or `OrderedDict` with `move_to_end` for a hand-made one (Q273); lookups by key → `dict`; sorted container with fast inserts → the third-party `sortedcontainers` package. → *More:* Ch 6 §6.12 — When to use what

---

4. Functions, arguments, scope and closures

Q69. ⋆ What are `*args` and `**kwargs`? In a function definition, `*args` collects extra positional arguments into a tuple and `**kwargs` collects extra keyword arguments into a dict (the names are just a convention). At a call site, `*seq` and `**mapping` unpack values into arguments. Typical use: wrappers and decorators that pass arguments through unchanged, `def wrapper(*args, **kwargs): return func(*args, **kwargs)`. → *More:* Ch 20 Q18 · Ch 7 §7.2 — Function arguments

Q70. ⋆ Does Python pass arguments by value or by reference? By object reference ("call by sharing"): the parameter becomes a new name for the caller's object. If the function *mutates* a mutable object (`lst.append(1)`), the caller sees it; if it *rebinds* the parameter (`lst = []`), the caller does not. With immutable objects such as `int` and `str`, a function therefore can never change the caller's value. → *More:* Ch 20 Q4 · Ch 7 §7.2

Q71. ⋆ Why shouldn't you use a mutable default argument such as `def f(x, items=[])`? Default values are evaluated **once**, when the `def` statement runs, and the same object is reused on every call. So appending to `items` accumulates data across calls. Fix: use `None` as the default and create the list inside: `if items is None: items = []`. The same applies to `{}`, `set()` and objects like `datetime.now()` (evaluated once at definition time). → *More:* Ch 20 gotcha 1 · Ch 7 §7.2

Q72. What do `/` and `*` mean in a function signature? What is the order of parameter kinds? Parameters before `/` are positional-only (cannot be passed by name); parameters after `*` (or after `*args`) are keyword-only (must be passed as `name=value`). Full order: positional-only, `/`, normal, `*args` or `*`, keyword-only, `**kwargs`. Positional-only protects parameter names you may want to rename later (many built-ins such as `len(obj, /)` use it); keyword-only forces readable calls (`sorted(xs, key=..., reverse=True)`).

```
def f(a, b, /, c, *, d, **rest): ...
f(1, 2, 3, d=4)          # ok
f(1, b=2, c=3, d=4)      # TypeError: missing positional argument 'b'
                         # (b is positional-only, so b=2 just lands in **rest)
```

→ *More:* Ch 7 §7.2

Q73. Can a parameter without a default follow one with a default? Not among positional parameters: `def f(a=1, b):` is a `SyntaxError`, because a positional argument could not tell which slot to fill. It is allowed for keyword-only parameters: `def f(a=1, *, b):` is fine since `b` must be passed by name. → *More:* Ch 7 §7.2

Q74. ⋆ What is a lambda function? What are its limitations? `lambda args: expression` creates a small anonymous function whose body is a single expression (no statements, no annotations). It is handy for short callbacks: `sorted(words, key=lambda w: w.lower())`. Do not assign a lambda to a name (use `def`, which gives a real name in tracebacks), and prefer `operator.itemgetter` / `attrgetter` or a named function when the logic grows. → *More:* Ch 20 Q24 · Ch 7 §7.5

Q75. What are first-class and higher-order functions? Functions are *first-class*: they are objects that can be assigned to variables, stored in data structures, passed as arguments and returned from other functions. A *higher-order function* takes or returns a function: `map`, `filter`, `sorted(key=...)`, `functools.reduce` and every decorator. This enables strategies as plain functions, callbacks and dispatch tables (`handlers = {"add": add, "sub": sub}`). → *More:* Ch 7 §7.5 — First-class and higher-order functions

Q76. ⋆ Explain `map()`, `filter()` and `reduce()`. Should you use them or comprehensions? `map(f, xs)` lazily applies `f` to each item; `filter(pred, xs)` lazily keeps items where `pred` is truthy; `functools.reduce(f, xs, init)` folds a sequence into one value (`reduce(operator.mul, xs, 1)` is a product). In Python 3, `map` and `filter` return iterators. Comprehensions are usually clearer, especially if you would need a `lambda`; `map(str, xs)` with an existing function is fine. For common reductions use `sum`, `min`, `max`, `any`, `all`, `math.prod`. → *More:* Ch 20 Q25 · Ch 11 §11.7

Q77. ⋆ What is a namespace? What is the LEGB rule? A namespace is a mapping from names to objects; each module, function call and class body has its own, plus the built-ins namespace. When Python looks up a name it searches **L**ocal (current function), **E**nclosing (outer functions, for nested functions), **G**lobal (the module), then **B**uilt-ins (`len`, `print`), and raises `NameError` if none has it. Class bodies are *not*

an enclosing scope for methods: inside a method you must write `self.x` or `ClassName.x`. → *More:* Ch 20 Q20 · Ch 7 §7.3 — Scope and namespaces

Q78. What do `global` and `nonlocal` do? Assigning to a name inside a function normally creates a local variable. `global x` makes assignments in the function rebind the module-level `x`; `nonlocal x` makes them rebind `x` in the nearest enclosing function (needed for counters in closures). Reading an outer name needs neither keyword. Mutating an outer object (`cache.append(1)`) also needs neither; only rebinding does. Prefer returning values over `global`. → *More:* Ch 7 §7.3

Q79. Why does this raise `UnboundLocalError`?

```
count = 0
def increment():
    count += 1      # UnboundLocalError
```

The compiler sees an assignment to `count` inside the function and marks `count` as local for the *whole* function body. `count += 1` then reads the local before it has a value. Fix: `global count` (or better, pass and return the value, or use a class or closure with `nonlocal`). → *More:* Ch 7 §7.3

Q80. ⋆ What is a closure? Give a use case. A closure is an inner function that remembers variables from the enclosing function's scope after the outer function has returned. Those variables live in *cells* (`inner.__closure__`). Uses: function factories (`make_multiplier(3)`), decorators, callbacks with configuration, and simple private state.

```
def make_counter():
    n = 0
    def inc():
        nonlocal n
        n += 1
        return n
    return inc
c = make_counter(); c(); print(c())   # 2
```

→ *More:* Ch 20 Q19 · Ch 7 §7.4 — Closures

Q81. ⋆ What is late binding in closures? Why does `[lambda: i for i in range(3)]` give 2, 2, 2? Closures look up a variable's value when they are *called*, not when they are created. All three lambdas share the same `i`, which is 2 when the loop ends. Fix: bind the current value as a default argument, `lambda i=i: i`, or use `functools.partial`. → *More:* Ch 20 gotcha 2 · Ch 7 §7.4

Q82. What is `functools.partial`? `partial(func, *args, **kwargs)` returns a new callable with some arguments pre-filled: `int_from_binary = partial(int, base=2)`, then `int_from_binary("101") == 5`. Unlike a lambda it is picklable (when `func` is), inspectable (`.func`, `.args`, `.keywords`) and does not suffer from late binding. Common with callbacks and `map`. → *More:* Ch 7 §7.6

Q83. How does a function return multiple values? It returns one tuple, which the caller unpacks: `return total, count` then `t, c = f()`. For more than two or three values, return a `NamedTuple` or dataclass so the fields have names. A function with no `return` (or a bare `return`) returns `None`. → *More:* Ch 7 §7.1

Q84. Does Python support function overloading? Not by signature: a second `def` with the same name simply replaces the first. Alternatives: default and keyword arguments, `*args`, `functools.singledispatch` (chooses an implementation by the type of the first argument; `singledispatchmethod` for methods), or `@classmethod` alternative constructors. `typing.overload` only describes several signatures to type checkers; you still write one implementation. → *More:* Ch 7 §7.6 · Ch 13

Q85. What is recursion in Python? What is the recursion limit? Does Python optimise tail calls? A recursive function calls itself on a smaller input until it reaches a base case. CPython limits the call depth (default 1000; `sys.getrecursionlimit()` / `setrecursionlimit()`) and raises `RecursionError` beyond it. Python does **not** do tail-call optimisation (reusing the stack frame for a call in tail position), by design, to keep full tracebacks. For deep recursion, convert to a loop with an explicit stack, or memoise with `@functools.cache` to cut repeated calls. → *More:* Ch 7 §7.7 — Recursion · Ch 19 §19.10

Q86. What are function annotations and docstrings used for at runtime? Annotations (`def f(x: int) -> str`) are metadata stored for tools; Python does not enforce them. In 3.14 they are evaluated lazily, on first access via `f.__annotations__` or `annotationlib.get_annotations(f)`. Docstrings are stored in

`f.__doc__`. Frameworks read both: FastAPI builds request validation from annotations, and `help()` shows docstrings. → *More:* Ch 7 §7.8 · Ch 13 §13.14

Q87. What does the single underscore `_` mean in Python? By convention: a throwaway variable (`for _ in range(3)`, `_, ext = os.path.splitext(p)`); a leading underscore marks a name as internal (`_helper`; also skipped by `from m import *`); in the interactive interpreter `_` holds the last result; in `match` statements `case _:` is the wildcard; and `1_000_000` uses underscores as digit separators. A trailing underscore avoids clashing with a keyword (`class_`). → *More:* Ch 10 §10.9

---

5. OOP, dunder methods and the MRO

Q88. ⋆ What is a class and an object? What is `self`? A class is a blueprint that defines attributes and methods; an object (instance) is one concrete thing built from it, with its own state. `self` is the instance the method was called on. Python passes it explicitly as the first parameter: `obj.method(1)` is just `Class.method(obj, 1)`. The name `self` is a convention, not a keyword. → *More:* Ch 10 §10.1 — Classes and instances

Q89. ⋆ What are the four pillars of OOP, and how does Python support them? Encapsulation (bundling data with the methods that use it; Python relies on naming conventions and properties rather than access modifiers), **abstraction** (exposing a simple interface; ABCs and Protocols), **inheritance** (a class reuses and extends another; Python allows multiple inheritance), and **polymorphism** (the same call works on different types; achieved by method overriding and, above all, duck typing). → *More:* Ch 10

Q90. ⋆ What is the difference between `__init__` and `__new__`? Which is called first when you create an object? `MyClass(args)` calls `type.__call__`, which first calls `__new__(cls, ...)` to **create** the instance, then, if `__new__` returned an instance of the class, calls `__init__(self, ...)` to **initialise** it. `__init__` must return `None`. Override `__new__` only for subclasses of immutable types (`int`, `str`, `tuple`), singletons/instance caches, or metaclass work. → *More:* Ch 20 Q26 · Ch 10 §10.1

Q91. ⋆ What is the difference between a class variable and an instance variable? A class variable is defined in the class body and shared by all instances; an instance variable is set on `self` (usually in `__init__`) and belongs to one object. Reading `obj.x` looks in the instance first, then the class. Assigning `obj.x = ...` always creates an instance attribute that *shadows* the class one. Danger: a mutable class variable (`items = []`) is shared, so appending through one instance changes it for all. → *More:* Ch 20 gotcha 8 · Ch 10 §10.1

Q92. ⋆ What is the difference between an instance method, `@classmethod` and `@staticmethod`? An instance method receives `self` and works with one object. A class method receives `cls` (the class it was called on, including subclasses), so it is the natural tool for alternative constructors (`Date.from_string("2026-10-14")`). A static method receives nothing automatically; it is a plain function placed in the class namespace because it logically belongs there. → *More:* Ch 20 Q27 · Ch 10 §10.1

Q93. ⋆ What does `@property` do? Why not write getters and setters? `@property` turns a method into an attribute that is computed on access; `@x.setter` and `@x.deleter` add validation on assignment and deletion. You can start with a plain public attribute and later turn it into a property without changing any calling code, so Java-style `get_x()`/`set_x()` methods are unnecessary. `functools.cached_property` computes once and stores the result on the instance. → *More:* Ch 20 Q28 · Ch 10 · Ch 12 §12.2

Q94. ⋆ How do you make a variable private in Python? What is name mangling? Python has no true private members. Convention: `_name` means "internal, don't touch from outside". A double leading underscore `__name` (without trailing underscores) triggers **name mangling**: inside class `Account`, `self.__balance` is stored as `_Account__balance`. Its purpose is to avoid accidental clashes with names in subclasses, not security; the attribute is still reachable by its mangled name. → *More:* Ch 10 §10.9 — Encapsulation conventions

Q95. What is inheritance? What types of inheritance does Python support? Inheritance lets a subclass reuse and extend a base class: `class Dog(Animal):`. Python supports single, multilevel (A → B → C), hierarchical (several children of one parent), multiple (`class C(A, B)`) and therefore hybrid/diamond inheritance. Every class ultimately inherits from `object`. A subclass overrides a method by defining it again and calls the parent's version with `super()`. → *More:* Ch 10 §10.4 — Inheritance and super()

Q96. ⋆ What does `super()` do? Why is it better than calling `Parent.method(self)`? `super()` returns a proxy that finds the *next* class in the method resolution order (MRO) of the instance's type, not simply "my parent". When every class in a hierarchy calls `super().__init__(**kwargs)`, each class's method runs exactly once, even in diamond hierarchies (cooperative multiple inheritance). Hard-coding `Parent.__init__(self)` skips classes or calls the shared base twice. → *More:* Ch 10 §10.4–10.5

Q97. ⋆ What is the MRO? How does Python resolve the diamond problem? The method resolution order is the list of classes Python searches for an attribute, available as `Class.__mro__` or `Class.mro()`. Python computes it with **C3 linearisation**: a child comes before its parents, the left-to-right order of bases is kept, and each class appears once. In the diamond `class D(B, C)` with `B(A)` and `C(A)`, the MRO is `D, B, C, A, object`, so `A` is visited once, after both `B` and `C`. → *More:* Ch 20 Q29 · Ch 10 §10.5 — Multiple inheritance and the MRO

Q98. What happens if C3 cannot build a consistent MRO? Class creation fails with `TypeError: Cannot create a consistent method resolution order (MRO)`. Example: `class X(A, B)` and `class Y(B, A)`, then `class Z(X, Y)`: X requires A before B and Y requires B before A, which cannot both hold. Fix the base order, or replace some inheritance with composition. → *More:* Ch 10 §10.5

Q99. ⋆ What is the difference between `__str__` and `__repr__`? `__repr__` is the developer-facing representation: unambiguous, ideally valid Python that recreates the object (`Point(x=1, y=2)`); the REPL, debuggers and containers (`print([p])`) use it. `__str__` is the user-facing text used by `print()`, `str()` and f-strings, and falls back to `__repr__` if not defined. Always define `__repr__`; add `__str__` only if users need a different text. → *More:* Ch 20 Q30 · Ch 10 §10.2

Q100. ⋆ What are dunder (magic) methods? Name the common ones. "Dunder" (double underscore) methods are hooks Python calls to implement syntax and built-ins: the **data model**. Construction: `__new__`, `__init__`; representation: `__repr__`, `__str__`, `__format__`; comparison: `__eq__`, `__lt__`, `__hash__`; arithmetic: `__add__`, `__radd__`, `__iadd__`; containers: `__len__`, `__getitem__`, `__setitem__`, `__contains__`, `__iter__`; truthiness: `__bool__`; calling: `__call__`; context managers: `__enter__`, `__exit__`; attribute access: `__getattr__`, `__setattr__`. → *More:* Ch 20 Q31 · Ch 10 §10.2 — The data model

Q101. How do you overload operators in Python? What is `NotImplemented`? Define the matching dunder method: `__add__` for `+`, `__mul__` for `*`, `__eq__` for `==`, `__lt__` for `<`, `__getitem__` for `[]`. If your method cannot handle the other operand's type, **return** `NotImplemented` (a special singleton, not the exception `NotImplementedError`); Python then tries the reflected method on the other operand (`__radd__`), and raises `TypeError` only if both decline. Define `__iadd__` to make `+=` mutate in place.

```python
class Vec:
    def __init__(self, x, y): self.x, self.y = x, y
    def __add__(self, other):
        if not isinstance(other, Vec):
            return NotImplemented
        return Vec(self.x + other.x, self.y + other.y)
```

→ *More:* Ch 10 §10.3 — Operator overloading

Q102. If you override `__eq__`, what happens to `__hash__`? How do you make objects usable in sets? Defining `__eq__` without `__hash__` sets `__hash__` to `None`, making instances unhashable, because the default identity-based hash would break the rule "equal objects must have equal hashes". To use objects in sets and as dict keys, define `__hash__` from the same immutable fields used in `__eq__` (for example `hash((self.x, self.y))`), or use `@dataclass(frozen=True)`, which generates both. → *More:* Ch 20 Q13 · Ch 6 §6.2 · Ch 10 §10.10

Q103. How can you implement all comparison operators with little code? Define `__eq__` and one ordering method (say `__lt__`) and decorate the class with `functools.total_ordering`, which fills in `__le__`, `__gt__` and `__ge__`. Alternatively, `@dataclass(order=True)` generates all of them by comparing fields as tuples in declaration order. → *More:* Ch 10 §10.3 · Ch 12 §12.2

Q104. What does `__call__` do? It makes instances callable like functions: `obj(args)` runs `obj.__call__(args)`. Useful for objects that need state between calls (a counter, a rate limiter, a configured model), for class-based decorators, and for strategy objects. `callable(x)` checks whether something can be called. → *More:* Ch 10 §10.2 · Ch 12 §12.1

Q105. How do you make your own class behave like a container (support `len`, indexing, `in` and iteration)? Implement `__len__`, `__getitem__` (and `__setitem__`/`__delitem__` for mutability), `__contains__` and `__iter__`. With only `__getitem__` accepting integers from 0, Python can already iterate and test `in` (the legacy sequence protocol). The cleanest route is to inherit from `collections.abc.Sequence` or `MutableMapping`, implement the few abstract methods, and get the rest (`index`, `count`, `get`, `items`...) as mixin methods. → *More:* Ch 10 §10.2 · Ch 10 §10.7

Q106. What is polymorphism? What is the difference between method overloading and overriding? Polymorphism means one interface, many implementations: `shape.area()` works for any shape. **Overriding** is a subclass redefining an inherited method; Python fully supports it. **Overloading** (several methods with the same name but different parameter lists) is not supported directly: the last definition wins. Use default arguments or `singledispatch` instead (Q84). → *More:* Ch 10 §10.4

Q107. ⋆ What is an abstract base class? How do you create one? An ABC defines an interface that subclasses must implement. Inherit from `abc.ABC` and mark methods with `@abstractmethod`; trying to instantiate a class that still has abstract methods raises `TypeError`. `collections.abc` provides ready-made ABCs (`Iterable`, `Mapping`, `Sequence`) that also work with `isinstance` checks.

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...
```

→ *More:* Ch 20 Q33 · Ch 10 §10.7 — ABCs, duck typing and Protocols

Q108. What is the difference between an ABC and a `Protocol`? An ABC is **nominal**: a class conforms only if it inherits from (or is registered with) the ABC, and the check happens at runtime when instantiating. A `typing.Protocol` is **structural**: any class with the right methods conforms, without inheriting, and the check is done by a static type checker (mypy, pyright). Protocols fit duck typing and third-party classes you can't modify; ABCs fit when you want shared code and enforced implementation. → *More:* Ch 20 Q33 · Ch 13 §13.8 — Protocol

Q109. ⋆ What is a dataclass? What does it generate, and what are its main options? `@dataclass` reads the class's annotated fields and generates `__init__`, `__repr__` and `__eq__`. Options: `frozen=True` (immutable and hashable), `order=True` (comparison methods), `slots=True` (uses `__slots__`), `kw_only=True` (keyword-only constructor). Use `field(default_factory=list)` for mutable defaults and `__post_init__` for validation. Since 3.13, `copy.replace(obj, x=1)` makes a modified copy of a dataclass instance. → *More:* Ch 20 Q34 · Ch 10 §10.10 — dataclasses

Q110. When would you use a dataclass, a Pydantic model or a plain class? A dataclass for internal data records where types are trusted (no runtime validation). A Pydantic model where data comes from outside (JSON from an API, config files): it validates and converts types at runtime and serialises back to JSON. A plain class when behaviour (methods, invariants) matters more than the data fields. → *More:* Ch 13 §13.14 — Runtime vs static, and pydantic

Q111. ⋆ What is `__slots__`? When would you use it? `__slots__ = ("x", "y")` replaces the per-instance `__dict__` with fixed storage for just those attributes. This saves memory (noticeable with millions of objects) and speeds up attribute access slightly, but you cannot add other attributes, and multiple inheritance with slots needs care. `@dataclass(slots=True)` generates it for you. → *More:* Ch 20 Q32 · Ch 10 §10.12 — **slots**

Q112. Composition vs inheritance: which do you prefer and why? Inheritance models an "is-a" relationship and couples the subclass tightly to the base class's internals; composition models "has-a" (an object holds other objects and delegates to them) and keeps parts replaceable and testable. The usual advice is "prefer composition over inheritance": inherit for genuine subtype relationships or framework hooks, compose everything else. → *More:* Ch 10 §10.8 — Composition vs inheritance

Q113. What is a mixin? A mixin is a small class that adds one capability (for example `JsonMixin` adding `to_json()`, or `LoggingMixin`) and is not meant to be instantiated alone. You combine it with a main base class through multiple inheritance: `class User(JsonMixin, Model):`. Mixins should be stateless or cooperative (call `super()`), and are listed before the main base. → *More:* Ch 10 §10.6 — Mixins

Q114. ⋆ How do you implement a singleton in Python? The simplest is a **module**: modules are imported once and cached, so a module-level object is a natural singleton. Other options: override `__new__` to return

a stored instance, a class decorator that caches the instance, or a metaclass. Interviewers also like the caveat: singletons are global state, which makes testing harder; dependency injection is often better.

```
class Config:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

→ *More:* Ch 10 §10.13 — Pythonic design patterns

Q115. What is an `Enum` and why use it instead of constants? `enum.Enum` defines a fixed set of named members (`Color.RED`) with `.name` and `.value`. Members are singletons, iterable, compared by identity, and type checkers can verify you handled every case. Variants: `IntEnum` / `StrEnum` (also behave as int or str), `Flag` (combinable bit flags), and `auto()` for automatic values. They prevent typos and invalid values that plain string constants allow. → *More:* Ch 10 §10.11 — Enums

Q116. What is the difference between `isinstance()` and `type() ==` ? `isinstance(obj, Base)` is true for instances of `Base` *and its subclasses* (and for ABC registrations), and accepts a tuple or a union of types (`isinstance(x, int | float)`). `type(obj) is Base` matches only the exact class. Prefer `isinstance` (or better, duck typing); `issubclass` does the same check for classes. → *More:* Ch 10 §10.7

Q117. What is the difference between `__getattr__` and `__getattribute__` ? `__getattribute__` runs on **every** attribute access; overriding it is rarely needed and easily causes infinite recursion (call `object.__getattribute__(self, name)` inside). `__getattr__` runs only when normal lookup **fails**, which makes it the safe hook for proxies, delegation and lazy attributes. `__setattr__` and `__delattr__` intercept assignment and deletion. → *More:* Ch 12 §12.5 — **getattr** vs **getattribute**

Q118. What is a descriptor? An object that defines `__get__` (and optionally `__set__` / `__delete__`) and is stored as a class attribute; accessing that attribute through an instance calls those methods. A *data* descriptor (has `__set__`) takes priority over the instance `__dict__`; a *non-data* one does not. `property`, `classmethod`, `staticmethod` and ordinary functions (which become bound methods through `__get__`) are all descriptors, as are ORM fields. `__set_name__` tells a descriptor its attribute name. → *More:* Ch 20 Q35 · Ch 12 §12.4 — Descriptors

Q119. ⋆ What is a metaclass? When would you use one? A metaclass is the class of a class: it controls how classes are created (the default is `type`, so `type(int) is type`). `type(name, bases, namespace)` even creates classes dynamically. Uses: frameworks that register or transform classes at definition time (Django models, `ABCMeta`, `EnumType`). For most needs, a class decorator or `__init_subclass__` (a hook called whenever a subclass is defined) is simpler and preferred. "If you wonder whether you need a metaclass, you don't." → *More:* Ch 20 Q35 · Ch 12 §12.6 — Metaclasses and **init_subclass**

Q120. How do you access a parent class attribute that is overridden in the child? Use `super().method()` or `super().attr` inside the class; from outside, call it explicitly on the parent: `Parent.method(obj)`. For class attributes, `Parent.attr`. With multiple inheritance, remember `super()` follows the MRO, so it may reach a sibling class rather than the class you named in the bases. → *More:* Ch 10 §10.4

---

6. Iterators, generators and comprehensions

Q121. ⋆ What is the difference between an iterable, an iterator and a generator? An **iterable** can produce an iterator: it has `__iter__` (lists, strings, dicts, files, `range`). An **iterator** produces items one at a time with `__next__`, raises `StopIteration` when done, returns itself from `__iter__`, and can be consumed only once. A **generator** is an iterator created by a function containing `yield` (or by a generator expression); Python writes `__iter__` / `__next__` for you and saves the function's state between items. → *More:* Ch 20 Q22 · Ch 11 §11.1 — The iteration protocol

Q122. How does a `for` loop work internally? `for x in obj:` calls `iter(obj)` once to get an iterator, then repeatedly calls `next()` on it and binds the result to `x`, until the iterator raises `StopIteration`, which the loop catches silently and ends (running the `else` block, if any). This is why any object

implementing the protocol works in `for`, comprehensions, `list()`, `sum()` and unpacking. → *More:* Ch 3 §3.5 · Ch 11 §11.1

Q123. ⋆ What is the difference between `yield` and `return`? `return` ends the function and sends back one value. `yield` sends back a value and **pauses** the function, keeping its local variables; the next `next()` resumes right after the `yield`. A function containing `yield` returns a generator object when called and runs no code until the first `next()`. A `return value` inside a generator ends it and stores the value in `StopIteration.value`. → *More:* Ch 20 Q23 · Ch 11 §11.2 — Generators and yield

Q124. ⋆ What are the advantages of generators? Give a real use case. They are **lazy**: they produce items on demand, so memory stays constant regardless of how many items there are, the first result arrives immediately, and they can represent infinite sequences. Use cases: reading a huge log file line by line, streaming rows from a database or API page by page, and pipelines of generator stages (`read → parse → filter → aggregate`) that process data without building intermediate lists. → *More:* Ch 11 §11.4 — Lazy evaluation and memory

Q125. ⋆ What is the difference between a list comprehension and a generator expression? `[x * x for x in xs]` builds the whole list in memory immediately; `(x * x for x in xs)` returns a lazy generator that computes values one at a time and can be consumed once. Use the generator form when you only iterate once or feed a function like `sum(x * x for x in xs)` (the extra parentheses can be dropped when it is the only argument); use a list when you need indexing, `len`, or several passes. → *More:* Ch 20 §20.1 · Ch 11 §11.6

Q126. What does `yield from` do? `yield from sub` yields every value from the sub-iterator `sub`, and for generators it also forwards `send()` and `throw()` calls and returns the sub-generator's return value. It replaces `for v in sub: yield v`, and is handy for recursive generators such as flattening nested lists or walking trees. → *More:* Ch 11 §11.3 — yield from, send() and close()

Q127. What do a generator's `send()`, `throw()` and `close()` methods do? `gen.send(value)` resumes the generator and makes the paused `yield` expression evaluate to `value` (the first call must be `send(None)` or `next()`). `gen.throw(exc)` raises an exception at the paused `yield`. `gen.close()` raises `GeneratorExit` inside it so `finally` blocks run. These turn generators into simple coroutines; today `async def` is used for real coroutines. → *More:* Ch 11 §11.3

Q128. How do you write your own iterator class? Implement `__iter__` returning `self` and `__next__` returning the next value or raising `StopIteration`. For a reusable iterable, make `__iter__` return a *new* iterator (often by making `__iter__` itself a generator function).

```
class Countdown:
    def __init__(self, start): self.n = start
    def __iter__(self): return self
    def __next__(self):
        if self.n <= 0:
            raise StopIteration
        self.n -= 1
        return self.n + 1
print(list(Countdown(3)))   # [3, 2, 1]
```

→ *More:* Ch 11 §11.1

Q129. What happens if a generator raises `StopIteration` itself? Since Python 3.7 (PEP 479), a `StopIteration` that escapes from inside a generator body is converted into `RuntimeError`, because it would otherwise silently end the loop and hide bugs. Inside a generator, use `return` to finish. A common trigger is calling `next(it)` on an exhausted iterator inside a generator; use `next(it, default)` instead. → *More:* Ch 11 §11.2

Q130. Is a list comprehension faster than a `for` loop with `append`? Usually yes, modestly (often 10–30 %): the comprehension avoids looking up and calling `lst.append` on each iteration and runs a specialised loop. Since 3.12 comprehensions are also inlined (PEP 709, no hidden function call). But choose them for readability; if the body needs several statements or side effects, a plain loop is clearer. For numeric work, NumPy vectorisation beats both by far. → *More:* Ch 11 §11.6 — Comprehensions, best practices · Ch 17 §17.7

Q131. How do nested comprehensions work? When should you avoid comprehensions? Clauses read left to right like nested loops: `[(i, j) for i in range(3) for j in range(i)]` equals an outer `for i` with an inner

`for j`. Flatten: `[x for row in matrix for x in row]`; transpose: `[list(col) for col in zip(*matrix)]`. Avoid comprehensions with more than two loops or complex conditions, and never use them only for side effects (`[print(x) for x in xs]`). → *More:* Ch 5 §5.6 · Ch 11 §11.6

Q132. Which `itertools` functions should you know? `chain` (concatenate iterables), `islice` (slice any iterator), `count` / `cycle` / `repeat` (infinite), `product`, `permutations`, `combinations`, `combinations_with_replacement` (combinatorics), `accumulate` (running totals), `groupby` (groups *consecutive* equal keys, so sort first), `pairwise` (3.10, adjacent pairs), `batched` (3.12, chunks of n), `zip_longest`, `takewhile` / `dropwhile`, `starmap`, `tee` (split one iterator in several). → *More:* Ch 11 §11.5 — itertools

Q133. What do `enumerate`, `zip`, `any` and `all` do? What does `zip(strict=True)` add? `enumerate(xs, start=0)` yields `(index, item)` pairs; `zip(a, b)` yields tuples of items in parallel and stops at the shortest input; `zip(a, b, strict=True)` (3.10+) raises `ValueError` if lengths differ, catching silent data loss. `any(it)` is true if any item is truthy, `all(it)` if all are; both short-circuit, and `all([])` is `True`, `any([])` is `False`. → *More:* Ch 11 §11.7 · Ch 20 §20.1

Q134. Is `range` a generator? What does it return in Python 3? No. `range` is a lazy, immutable **sequence**: it computes items on demand, but supports `len`, indexing, slicing, `in` (O(1) for integers) and can be iterated many times. A generator supports none of those and is single-use. → *More:* Ch 3 §3.6 · Ch 20 rapid-fire round

---

7. Decorators, context managers and metaprogramming

Q135. ⋆ What is a decorator? How does it work? A decorator is a callable that takes a function (or class) and returns a replacement, usually a wrapper that adds behaviour before or after calling the original. `@decorator` above `def f` is shorthand for `f = decorator(f)`, applied once when the function is defined. Uses: logging, timing, caching, retries, access control, registering routes (`@app.get("/")`). → *More:* Ch 20 Q21 · Ch 7 §7.6 · Ch 12 §12.1 — Decorators

Q136. ⋆ Write a decorator that measures a function's execution time.

```python
import functools, time
def timed(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            print(f"{func.__name__} took {time.perf_counter() - start:.4f}s")
    return wrapper
```

`perf_counter` is the right clock for measuring durations; `*args, **kwargs` forwards any arguments; `finally` reports even if the function raises. → *More:* Ch 12 §12.1

Q137. ⋆ Why use `functools.wraps`? Without it, the decorated function is replaced by `wrapper`, so `__name__`, `__doc__`, `__module__`, `__qualname__` and annotations show the wrapper's details, which confuses debugging, logging, documentation and frameworks that look up functions by name. `@functools.wraps(func)` copies that metadata and sets `wrapper.__wrapped__ = func`, so tools (and `inspect.signature`) can reach the original. → *More:* Ch 12 §12.1

Q138. ⋆ How do you write a decorator that takes arguments? Add one more level: an outer function receives the arguments and returns the actual decorator. `@retry(times=3)` first calls `retry(times=3)`, and the result decorates the function. So the structure is factory → decorator → wrapper (see the full example in Ch 20 Q21). → *More:* Ch 20 Q21 · Ch 12 §12.1

Q139. In what order are stacked decorators applied? Bottom-up when decorating, top-down when called. `@a` over `@b` over `def f` means `f = a(b(f))`: `b` wraps first; at call time `a`'s wrapper runs first, then `b`'s, then `f`. Example: `@bold @italic def hi()` produces `"<b><i>hi</i></b>"`. → *More:* Ch 12 §12.1

Q140. Can a class be a decorator? Can you decorate a class? Yes to both. A class with `__call__` can act as a decorator: `__init__` receives the function and `__call__` runs on each call, which is convenient when the decorator keeps state (call counts). A *class decorator* receives a class and returns it

modified or replaced: `@dataclass`, `@functools.total_ordering` and registration decorators work this way. → *More:* Ch 12 §12.1

Q141. What built-in and standard-library decorators do you know? `@property`, `@classmethod`, `@staticmethod`, `@functools.cache`, `@functools.lru_cache(maxsize=...)`, `@functools.cached_property`, `@functools.wraps`, `@functools.total_ordering`, `@functools.singledispatch`, `@dataclasses.dataclass`, `@contextlib.contextmanager`, `@abc.abstractmethod`, `@typing.override` (3.12) and `@typing.overload`. Third-party favourites: `@pytest.fixture`, `@app.route` / `@app.get`. → *More:* Ch 12 §12.2 — Everyday built-in decorators

Q142. ⋆ What is the difference between `functools.cache` and `lru_cache`? What are their pitfalls? Both memoise (store results by arguments and return the stored result on a repeat call). `@cache` (3.9) is unbounded; `@lru_cache(maxsize=128)` evicts the least recently used entry when full. Pitfalls: all arguments must be hashable; cached results are shared, so mutating a returned list corrupts the cache; an unbounded cache can grow forever; and decorating a *method* caches `self` too, keeping instances alive (a memory leak). Use `cache_info()` and `cache_clear()` to inspect and reset. → *More:* Ch 7 §7.6 · Ch 12 §12.2

Q143. ⋆ What is a context manager? How does the `with` statement work? A context manager is an object that sets something up and guarantees clean-up, implementing `__enter__` and `__exit__`. `with cm as x:` calls `cm.__enter__()` and binds its return value to `x`, runs the block, then always calls `cm.__exit__(exc_type, exc, tb)`, even after an exception or `return`. If `__exit__` returns a true value, the exception is suppressed. Typical uses: files, locks, database transactions, temporary changes. → *More:* Ch 20 Q37 · Ch 9 §9.8 · Ch 12 §12.3 — Context managers

Q144. Write a context manager in two ways.

```
class Timer:
    def __enter__(self):
        self.start = time.perf_counter(); return self
    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.start
        return False                    # don't suppress exceptions

@contextlib.contextmanager
def timer():
    start = time.perf_counter()
    try:
        yield
    finally:
        print(f"{time.perf_counter() - start:.3f}s")
```

The generator version runs code before `yield` on enter and after it on exit; the `try/finally` makes the exit code run even if the block raises. → *More:* Ch 12 §12.3

Q145. What helpers does `contextlib` provide? `contextmanager` (generator-based managers), `asynccontextmanager`, `suppress(FileNotFoundError)` (ignore specific exceptions), `closing(obj)` (call `.close()` on exit), `nullcontext()` (a do-nothing manager for optional `with`), `redirect_stdout`, `chdir` (3.11), and `ExitStack` (enter a variable number of managers and close all of them in reverse order). → *More:* Ch 12 §12.3 · Ch 9 §9.8

Q146. What is an async context manager? An object with `async def __aenter__` and `async def __aexit__`, used with `async with` inside a coroutine, so setting up and cleaning up can await I/O (opening a network connection, acquiring an `asyncio.Lock`, starting a `TaskGroup`). `contextlib.asynccontextmanager` builds one from an async generator. → *More:* Ch 12 §12.3 · Ch 16 §16.9

Q147. ⋆ What is monkey patching? When is it acceptable? Changing a module, class or object attribute at runtime (`requests.get = fake_get`). Acceptable in tests, ideally through `unittest.mock.patch` or pytest's `monkeypatch`, which restore the original automatically; occasionally for an emergency hotfix of a third-party bug. Risky in production code: global, invisible behaviour changes that break on library upgrades. → *More:* Ch 20 Q46 · Ch 12 §12.7 — Monkey patching

Q148. What are `getattr`, `setattr`, `hasattr` and the `inspect` module used for? They access attributes by *name* at runtime: `getattr(obj, "save", None)()` for dynamic dispatch, `setattr` to set from config, `hasattr` to check (it calls `getattr` and catches `AttributeError`). `inspect` examines live objects:

`inspect.signature(f)`, `getsource`, `getmembers`, `iscoroutinefunction`. Frameworks use these for plugin systems, dependency injection and serialisation. → *More:* Ch 12 §12.8 — Introspection and dynamic code

Q149. Why are `eval()` and `exec()` dangerous? What is the safe alternative? `eval` evaluates an expression string and `exec` runs statements; given untrusted input, they run arbitrary code (delete files, read secrets), and restricting their globals is not a reliable sandbox. To parse literal data use `ast.literal_eval` (only literals: numbers, strings, lists, dicts...), `json.loads` for JSON, and explicit dispatch tables instead of building code strings. → *More:* Ch 12 §12.8

Q150. What is `__init_subclass__`? Give a use case. A class method automatically called on a base class whenever a subclass is created, receiving the new subclass (and any keyword arguments from the class line). It covers most things people used metaclasses for: registering plugins (`registry[cls.__name__] = cls`), validating that subclasses define required attributes, or configuring subclasses (`class Json(Serializer, fmt="json")`). → *More:* Ch 12 §12.6

---

8. Exceptions and files

Q151. ⋆ How do you handle exceptions? Explain `try`, `except`, `else` and `finally`. Code that may fail goes in `try`. Each `except SomeError as e:` handles matching exceptions (checked top to bottom). `else` runs only if `try` raised nothing; put the code that should run on success but should *not* be protected there. `finally` always runs, whether an exception happened, was handled, or the block returned; use it (or `with`) for clean-up. → *More:* Ch 20 Q36 · Ch 9 §9.2

Q152. ⋆ Describe the exception hierarchy. Why is a bare `except:` bad? Everything derives from `BaseException`. Directly under it are `SystemExit`, `KeyboardInterrupt`, `GeneratorExit` and `Exception`; nearly all errors (`ValueError`, `KeyError`, `OSError` ...) derive from `Exception`. A bare `except:` (or `except BaseException:`) also catches Ctrl+C and `sys.exit()`, and hides real bugs. Catch the most specific exception you can handle; if you must catch broadly, use `except Exception` and log the traceback. → *More:* Ch 20 gotcha 11 · Ch 9 §9.1 — The hierarchy

Q153. Does the order of `except` blocks matter? Yes. Python uses the first `except` clause whose type matches (including base classes). Put specific exceptions before general ones: if `except Exception` comes first, a later `except ValueError` can never run. Catch several types in one clause with a tuple: `except (KeyError, IndexError) as e:`. Since 3.14 (PEP 758) the parentheses may be dropped when there is no `as` clause: `except KeyError, IndexError:`. → *More:* Ch 9 §9.2

Q154. ⋆ How do you create a custom exception? Subclass `Exception` (or a more specific built-in such as `ValueError`), usually with a descriptive name ending in `Error`. A library typically defines one base exception and specific subclasses so callers can catch at the level they want. Add fields if callers need data.

```
class PaymentError(Exception): pass
class CardDeclined(PaymentError):
    def __init__(self, code: str):
        super().__init__(f"card declined: {code}")
        self.code = code
```

→ *More:* Ch 9 §9.3 — Raising, re-raising and custom exceptions

Q155. What is the difference between `raise`, `raise e` and `raise NewError(...) from e`? A bare `raise` inside an `except` re-raises the current exception with its original traceback (the usual way to log and propagate). `raise e` also re-raises it, adding the current line to the traceback. `raise NewError(...) from e` raises a different exception and records `e` as its explicit cause (`__cause__`), shown as "The above exception was the direct cause of...". Raising inside `except` without `from` still links them implicitly (`__context__`); `from None` hides the original. → *More:* Ch 9 §9.4 — Exception chaining and notes

Q156. What happens if both `try` and `finally` contain `return`? The `finally` block's `return` wins and silently discards the `try`'s return value, and even swallows an exception that was propagating. That is why Python 3.14 (PEP 765) emits a `SyntaxWarning` for `return`, `break` or `continue` that would leave a `finally` block. Keep `finally` for clean-up only. → *More:* Ch 9 §9.2

Q157. What are `ExceptionGroup` and `except*`? `ExceptionGroup` (3.11) bundles several unrelated exceptions raised together, for example by concurrent tasks in an `asyncio.TaskGroup`. `except* ValueError as eg:` handles the matching *part* of a group (`eg` is a sub-group) and lets the rest continue to propagate; several `except*` clauses can each run. Also new in 3.11: `e.add_note("context")` attaches extra text shown in the traceback. → *More:* Ch 9 §9.6 — ExceptionGroup and except*

Q158. What is EAFP vs LBYL? Which is more Pythonic? EAFP, "easier to ask forgiveness than permission": just try the operation and handle the exception (`try: v = d[k] except KeyError: ...`). LBYL, "look before you leap": check first (`if k in d:`). Python favours EAFP: it is often faster when failure is rare, and it avoids race conditions where the situation changes between the check and the action (a file deleted after `os.path.exists`). → *More:* Ch 9 §9.5 — EAFP vs LBYL · Ch 20 §20.1

Q159. When should you use `assert`? For internal sanity checks that should never fail if the code is correct (invariants during development and in tests). Never for validating user input or enforcing security: running Python with `-O` removes all `assert` statements. Raise a proper exception (`ValueError`, `TypeError`) for input errors. Also beware `assert (cond, "msg")`: a non-empty tuple is always true. → *More:* Ch 9 §9.7 — assert

Q160. ⋆ How do you read a very large file that does not fit in memory? Iterate over the file object, which yields one line at a time with constant memory: `with open(path, encoding="utf-8") as f: for line in f: ...`. For binary data read fixed-size chunks: `while chunk := f.read(1 << 20):`. Chain generators to process lines lazily; use `csv.reader` on the open file for CSVs, `pandas.read_csv(..., chunksize=...)` for dataframes, and `mmap` for random access. Avoid `f.read()` and `f.readlines()`, which load everything. → *More:* Ch 9 §9.9 — File I/O · Ch 11 §11.4

Q161. What are the file modes? Why should you always use `with open(...)`? `"r"` read (default), `"w"` write (truncates), `"a"` append, `"x"` create (fails if the file exists), `"+"` read and write; add `"b"` for binary (bytes) instead of text. `with` closes the file even when an exception occurs, flushing buffered writes; without it, files may stay open (leaking file handles) or lose data. Always pass `encoding="utf-8"` for text, since the default depends on the platform. → *More:* Ch 9 §9.9 · Ch 20 §20.1

Q162. ⋆ What is the difference between `json` and `pickle`? Is pickle safe? `json` writes a text format of basic types (dict, list, str, number, bool, null) that any language can read; it is safe to load. `pickle` writes a Python-only binary format that can store almost any Python object, but **loading a pickle can execute arbitrary code**, so never unpickle untrusted data. Use JSON (or formats like Parquet) for exchange and storage, pickle only between trusted Python processes. → *More:* Ch 20 Q39 · Ch 9 §9.11 — csv, json, pickle

Q163. How do you serialise an object that `json.dumps` does not support, such as a `datetime` or a dataclass? Convert it first (`dataclasses.asdict(obj)`, `dt.isoformat()`), or pass `default=` a function that converts unknown types: `json.dumps(data, default=str)` or a custom function that handles `datetime`, `Decimal` and sets. For loading, `object_hook=` turns dicts back into objects. Libraries such as Pydantic do this both ways with validation. → *More:* Ch 9 §9.11 · Ch 14 §14.9

Q164. `pathlib` vs `os.path`: which should you use? `pathlib.Path` (3.4+) represents paths as objects with operators and methods: `Path("data") / "raw" / "a.csv"`, `.exists()`, `.suffix`, `.stem`, `.read_text()`, `.glob("*.csv")`, `.mkdir(parents=True, exist_ok=True)`. It is cross-platform and more readable than string functions in `os.path`. Use `shutil` for copying and moving trees and `tempfile` for temporary files. → *More:* Ch 9 §9.10 — pathlib, shutil, tempfile

---

9. Modules, packaging, environments and type hints

Q165. ⋆ What is the difference between a module and a package? A module is a single `.py` file; its name is the file name. A package is a directory of modules (and sub-packages) imported with dotted names (`pkg.sub.mod`). A *regular* package contains an `__init__.py`, which runs on first import and can expose a public API; a directory without it is a *namespace package* that can span several directories. → *More:* Ch 20 Q7 · Ch 8 §8.3 — Packages

Q166. ⋆ What does `if __name__ == "__main__":` do? Python sets `__name__` to `"__main__"` in the file being run directly and to the module's import name when it is imported. Code under the guard therefore runs only when the file is executed as a script, not when it is imported (for example by tests). It is also

required with `multiprocessing` under the `spawn` and `forkserver` start methods, because child processes re-import the main module. → *More:* Ch 20 Q6 · Ch 8 §8.5

Q167. How does `import` work? What are `sys.path` and `sys.modules`? `import x` first checks the cache `sys.modules`; if `x` is there, it reuses it (so a module's top-level code runs once per process). Otherwise finders search the directories in `sys.path` (the script's directory, `PYTHONPATH`, the standard library, then `site-packages`), a loader creates the module object, executes its code, stores it in `sys.modules`, and binds the name. `importlib.reload(m)` re-executes a module, but existing references keep the old objects. → *More:* Ch 8 §8.2 — How import really works

Q168. What is the difference between absolute and relative imports? An absolute import names the full path from the project root (`from shop.billing import invoice`). A relative import uses dots relative to the current package (`from .billing import invoice`, `from .. import utils`) and works only inside packages, not in a file run directly as a script. PEP 8 recommends absolute imports; run modules inside packages with `python -m pkg.mod`. → *More:* Ch 8 §8.4 — Absolute vs relative imports

Q169. ⋆ What causes a circular import, and how do you fix it? Module A imports B while B (directly or indirectly) imports A. Since A is only partly executed when B runs, B's `from a import thing` fails with `ImportError: cannot import name ... (most likely due to a circular import)`. Fixes: move shared code into a third module; import the module instead of names (`import a` then use `a.thing` at call time); move the import inside the function that needs it; or use `if TYPE_CHECKING:` for imports only needed by type hints. → *More:* Ch 20 gotcha 9 · Ch 8 §8.6

Q170. What does `__all__` do? Why avoid `from module import *`? `__all__` is a list of names that `from module import *` imports (otherwise all names not starting with `_`), and it documents the public API. Star imports are discouraged: they hide where names come from, can silently overwrite existing names, and confuse linters. Import what you use explicitly. → *More:* Ch 8 §8.3

Q171. ⋆ What is a virtual environment, and why use one? A directory (`python -m venv .venv`) containing its own Python executable link and `site-packages`, so each project gets its own isolated set of package versions without conflicts or touching the system Python. Activate it (`source .venv/bin/activate`) or call its interpreter directly. Never commit the folder; commit the dependency list instead. → *More:* Ch 20 Q8 · Ch 8 §8.7 — pip and virtual environments

Q172. What is the difference between `requirements.txt` and `pyproject.toml`? What are uv and Poetry? `requirements.txt` is a plain list of packages (often pinned with `==`) that `pip install -r` installs. `pyproject.toml` (PEP 518/621) is the standard project file: metadata, dependencies with version ranges, the build backend and tool settings (ruff, pytest, mypy). Tools such as **uv** (very fast, written in Rust) and **Poetry** manage the virtual environment, resolve dependencies and write a **lock file** that pins exact versions of everything, including indirect dependencies, for reproducible installs. → *More:* Ch 8 §8.8 — requirements.txt vs pyproject.toml; uv and Poetry

Q173. What is a wheel? What does `pip install -e .` do? A wheel (`.whl`) is a pre-built distribution: a zip file installed by unpacking, with no build step, and, for packages with C code, compiled for a specific platform. An sdist is the source archive that must be built. `pip install -e .` installs your project in *editable* mode: the environment points at your source folder, so code edits take effect without reinstalling. → *More:* Ch 8 §8.9 — Project layout and publishing basics

Q174. ⋆ What are type hints? Does Python enforce them? Annotations such as `def area(r: float) -> float:` and `names: list[str] = []` that document expected types. The interpreter does **not** check them; static type checkers (mypy, pyright) and IDEs use them to find bugs before running, and libraries such as Pydantic and FastAPI read them to validate data at runtime. Typing is gradual: unannotated code is treated as `Any`. → *More:* Ch 20 Q44 · Ch 13 §13.1–13.2

Q175. What is the difference between `Optional[X]`, `X | None` and a parameter with a default? `Optional[X]` and `X | None` (3.10+ syntax, preferred) both mean "an `X` or `None`". This is about the *type*, not about being optional to pass: a parameter is optional to pass only if it has a default. `def f(x: int | None)` must still be called with an argument; `def f(x: int | None = None)` may be called without one. → *More:* Ch 13 §13.4 — Optional, Union and X | None

Q176. What is the difference between `Any` and `object` in type hints? `Any` switches type checking off: every operation on it is allowed and it is compatible with every type. `object` is the top type: any value can be assigned to it, but you can only do what every object supports, so the checker forces you

to narrow it (`isinstance`) first. Prefer `object` (or a precise type) when you mean "anything"; `Any` is an escape hatch. → *More:* Ch 13 §13.5 — Any and object

Q177. How do you write generic functions and classes? What changed in 3.12? A type variable stands for "some type, the same in each place". Since 3.12 (PEP 695) you write `def first[T](xs: list[T]) -> T:` and `class Stack[T]:` directly, and `type Pair[T] = tuple[T, T]` defines an alias. Older code uses `T = TypeVar("T")` and `Generic[T]`. Bounds and constraints: `def biggest[T: Comparable](...)` or `[T: (int, float)]`. → *More:* Ch 13 §13.6 — TypeVar and generics · Ch 13 §13.12

Q178. What are `TypedDict`, `Literal`, `Final`, `NewType`, `Self` and `@override`? `TypedDict` describes dicts with specific keys and value types; `Literal["r", "w"]` restricts a value to given constants; `Final` marks a name that must not be reassigned; `NewType("UserId", int)` creates a distinct type for the checker at no runtime cost; `Self` (3.11) is the type of the current class in methods that return `self`; `@typing.override` (3.12) asks the checker to verify the method really overrides a base-class method. → *More:* Ch 13 §13.9–13.10

Q179. How do you type a callable or a decorator? `Callable[[int, str], bool]` is a function taking an `int` and a `str` and returning `bool`; `Callable[..., T]` accepts any arguments. To type a decorator that preserves the wrapped function's signature, use `ParamSpec`: `def logged[**P, R](f: Callable[P, R]) -> Callable[P, R]:`, with the wrapper declared `(*args: P.args, **kwargs: P.kwargs) -> R`. → *More:* Ch 13 §13.7 — Callable and ParamSpec

Q180. When are annotations evaluated? What changed in Python 3.14? Up to 3.13, annotations were evaluated when the `def` or `class` ran, so a reference to a class defined later had to be quoted (`"Node"`) unless the module used `from __future__ import annotations` (which stores them as strings). In 3.14 (PEP 649/749), evaluation is deferred: annotations are computed only when accessed, so forward references usually work without quotes, and the new `annotationlib` module can retrieve them as values, forward references or strings. → *More:* Ch 13 §13.2

Q181. mypy vs pyright vs Pydantic: what does each do? mypy and pyright are **static** type checkers: they read your code without running it and report type errors (pyright powers VS Code's Pylance and is faster; mypy is the original reference checker; both are commonly run in CI). Pydantic is a **runtime** library: it validates and converts incoming data according to type hints when the code runs. They solve different problems and are often used together. → *More:* Ch 13 §13.13–13.14

---

10. Memory model, internals and performance

Q182. ⋆ How is memory managed in Python? All objects live on a private heap managed by the interpreter. CPython frees objects mainly by **reference counting** (an object is freed the moment its count drops to zero) plus a **cyclic garbage collector** for groups of objects that reference each other. Small objects (up to 512 bytes) come from **pymalloc**, an allocator that keeps pools of fixed-size blocks to avoid many system `malloc` calls. You never free memory manually. → *More:* Ch 20 Q38 · Ch 17 §17.2 — Memory management and garbage collection

Q183. ⋆ What is reference counting? What are its pros and cons? Each object stores how many references point to it; assignments, container insertions and function arguments increase it, and deleting names or leaving scope decreases it. At zero the object is destroyed immediately. Pros: predictable, prompt clean-up (files closed as soon as unused). Cons: a small cost on every reference change, it cannot free reference cycles, and it makes multi-threading harder (the reason for the GIL). `sys.getrefcount(x)` shows the count (one higher, because of its own argument). → *More:* Ch 17 §17.2

Q184. ⋆ How does Python's garbage collector work? What are generations? The `gc` module finds reference cycles that refcounting cannot free (e.g., two objects pointing at each other, or an object referring to itself). It tracks container objects and is **generational**: new objects start in the young generation, which is scanned often; objects that survive are promoted to older generations, which are scanned rarely, because most objects die young. You can call `gc.collect()`, inspect `gc.get_threshold()`, or `gc.disable()` in special cases. Version note: 3.14.0–3.14.4 shipped an *incremental* collector, and 3.14.5 went back to the 3.13-style generational one. → *More:* Ch 17 §17.2

Q185. Can Python have memory leaks? How do you find them? Yes, in practice: references kept alive unintentionally. Typical causes: ever-growing module-level lists or dicts used as caches, unbounded

`lru_cache` (especially on methods), closures or callbacks holding large objects, reference cycles through objects with external resources, and C extensions that leak. Diagnose with `tracemalloc` (compare snapshots to see which lines allocate), `gc.get_objects()` / `objgraph`, and memory profilers; fix with bounded caches and `weakref`. → *More:* Ch 20 Q38 · Ch 17 §17.2, §17.6

Q186. What is a weak reference? A reference that does not increase the object's reference count, so it does not keep the object alive. `weakref.ref(obj)` returns a callable that gives the object or `None` once it is gone; `WeakValueDictionary` and `WeakKeyDictionary` make caches whose entries vanish when objects are no longer used elsewhere; `weakref.finalize` registers clean-up. Used for caches, observers and parent pointers in trees, to avoid leaks and cycles. → *More:* Ch 17 §17.2

Q187. Where are variables stored in Python: on the stack or the heap? All objects live on the heap. Each function call gets a *frame* holding its local variables, but those slots contain only references (pointers) to heap objects. That is why returning a local list is safe: the frame disappears, but the list lives on as long as something refers to it. → *More:* Ch 17 §17.1 — The CPython object model

Q188. ⋆ Why is `a = 256; b = 256; a is b` true but the same with large numbers can be false? CPython pre-creates and caches the integers from −5 to 256, so every 256 is the same object. Larger integers are usually created separately at runtime (`int("1000") is int("1000")` is `False`), although the compiler may reuse one constant object within the same code block, so results vary between the REPL and a script. This is an implementation detail: always compare numbers with `==`. → *More:* Ch 20 gotcha 4 · Ch 17 §17.3

Q189. What are immortal objects? Since 3.12 (PEP 683), certain objects such as `None`, `True`, `False`, small integers and some interned strings are *immortal*: their reference count is fixed at a special value and never changes, so they are never freed. This avoids writes to shared objects (good for multiple interpreters and the free-threaded build). It is why `sys.getrefcount(None)` returns a huge number. → *More:* Ch 17 §17.2

Q190. What is the difference between `sys.getsizeof()` and an object's real memory use? `sys.getsizeof(x)` returns only the size of the object itself, not of the objects it refers to: a list's size covers its pointer array, not its elements. For the total, walk the references (or use `pympler.asizeof`), or measure with `tracemalloc`. Typical sizes on 64-bit CPython: an empty list about 56 bytes, a small int 28 bytes. → *More:* Ch 17 §17.1

Q191. Why is `__del__` not a good way to release resources? `__del__` (the finaliser) runs when the object is destroyed, which with reference counting is usually immediate, but not if the object is part of a cycle (it waits for the GC), not on other Python implementations like PyPy, and not reliably at interpreter shutdown. Exceptions inside it are only printed. Use context managers (`with`) or `weakref.finalize` for deterministic clean-up. → *More:* Ch 17 §17.2

Q192. How do you find out why a Python program is slow? Measure before optimising. `cProfile` (`python -m cProfile -s cumtime app.py`) shows which functions take time; `pstats` or snakeviz explore the results; `timeit` compares small snippets; `line_profiler` times individual lines; `py-spy` samples a running process without changing it; `tracemalloc` finds memory hotspots. Then optimise the top hotspot only, and measure again. → *More:* Ch 20 Q40 · Ch 17 §17.6 — Profiling

Q193. ⋆ How do you make Python code faster? In order of payoff: a better algorithm or data structure (set or dict lookups instead of list scans, a heap instead of repeated sorting); push loops into C (built-ins like `sum`, `sorted`, `str.join`, `collections`, NumPy vectorisation); avoid repeated work (caching with `functools.cache`, hoisting work out of loops); reduce Python overhead in hot loops (local variables, comprehensions); use generators for memory; then parallelism (processes) or compiled code (Cython, Numba, Rust via PyO3), or PyPy. → *More:* Ch 17 §17.7 — Optimisation techniques

Q194. Why are local variables faster than global variables? Locals are stored in a fixed array in the frame and accessed by index (`LOAD_FAST`), while globals and built-ins require dictionary lookups (`LOAD_GLOBAL`, though modern CPython caches these). In very hot loops, binding a global or attribute to a local (`append = result.append`) can still save time, but readability usually matters more; the adaptive interpreter (3.11+) has narrowed the gap. → *More:* Ch 17 §17.7

Q195. What are Cython, Numba, PyPy and PyO3, and when would you use each? Cython compiles Python-like code with C type declarations into C extensions, good for wrapping C libraries or speeding up loops. **Numba** JIT-compiles numeric functions with a decorator (`@njit`), great for NumPy-heavy loops. **PyPy** is an alternative interpreter with a JIT that speeds up pure-Python programs without code changes. **PyO3** lets

you write extensions in Rust (as many modern tools like pydantic-core and polars do). Choose by bottleneck: numeric loops → Numba; existing C code → Cython; whole pure-Python app → PyPy; new performance-critical component → Rust. → *More:* Ch 17 §17.7

Q196. Does CPython have a JIT compiler? Since 3.13, CPython can be *built* with an experimental copy-and-patch JIT that translates frequently executed bytecode traces into machine code. It is off by default (enable with `PYTHON_JIT=1` on a JIT-enabled build; 3.14's official Windows and macOS installers include it), and gains are still modest. Most of the 3.11+ speed-ups came instead from the *specialising adaptive interpreter* (PEP 659), which rewrites bytecode into faster type-specific versions as code runs. → *More:* Ch 1 §1.8 · Ch 17

---

11. Concurrency: GIL, free-threading, threads, processes and asyncio

Q197. ⋆ What is the GIL (Global Interpreter Lock)? A lock inside the default CPython build that allows only one thread at a time to execute Python bytecode. It keeps reference counting and interpreter internals simple and makes single-threaded code fast. Consequences: threads do not speed up CPU-bound pure-Python code; they do help I/O-bound code because the GIL is released while waiting on I/O (and by many C extensions such as NumPy during heavy computations). The GIL does not make your code thread-safe. → *More:* Ch 20 Q41 · Ch 16 §16.6 — The GIL

Q198. ⋆ Why does CPython have a GIL, and is it being removed? What is free-threaded Python? It was the simplest way to make reference counting and the C API safe with threads, and removing it used to slow single-threaded code. PEP 703 introduced a separate **free-threaded** build without the GIL: experimental in 3.13 (`python3.13t`) and officially supported but still optional in 3.14 (PEP 779, `python3.14t`). The default `python3.14` still has the GIL. The free-threaded build costs roughly 5–10 % single-threaded speed and more memory; C extensions must be marked compatible, or the GIL is re-enabled when they are imported. Expect it to become the default only in a later phase. → *More:* Ch 16 §16.6 — Free-threaded CPython status

Q199. How can you tell whether the GIL is enabled at runtime? `sys._is_gil_enabled()` (3.13+) returns `False` on a free-threaded build running without the GIL. `sysconfig.get_config_var("Py_GIL_DISABLED")` tells whether the interpreter was *built* free-threaded. On such a build you can force the GIL back on with `PYTHON_GIL=1` or `-X gil=1`. → *More:* Ch 16 §16.6

Q200. What is the difference between concurrency and parallelism? Concurrency is *dealing* with many tasks at once by interleaving them (one cook switching between dishes while each simmers); parallelism is *doing* many tasks at the same instant on several CPU cores (several cooks). Threads and asyncio give concurrency; processes (and free-threaded threads) give parallelism. → *More:* Ch 16 §16.1 — Concurrency vs parallelism

Q201. ⋆ When do you use threading vs multiprocessing vs asyncio? CPU-bound work (number crunching, parsing, image processing) → `multiprocessing` or `ProcessPoolExecutor`: separate processes, each with its own GIL, true parallelism. I/O-bound work with blocking libraries and moderate concurrency (tens to hundreds of requests, file I/O) → `threading` or `ThreadPoolExecutor`. I/O-bound work with very many connections and async libraries (thousands of sockets, web servers) → `asyncio`. Numeric arrays → NumPy, which runs loops in C. → *More:* Ch 20 Q42 · Ch 16 §16.10 — Choosing the right tool

Q202. ⋆ What is the difference between a process and a thread? A process has its own memory space, interpreter and GIL; starting one is slow and data must be copied between processes (pickled), but a crash in one does not corrupt others. Threads live inside one process and share its memory, so they start fast and share data easily, but shared mutable data needs locks. In CPython's default build, only one thread runs Python code at a time. → *More:* Ch 16 §16.3, §16.7

Q203. ⋆ What is a race condition? Is `counter += 1` thread-safe? A race condition is a bug where the result depends on the unpredictable timing of threads. `counter += 1` is not atomic: it reads, adds and writes back in separate bytecodes, so two threads can read the same value and one increment is lost (with or without the GIL). Protect shared updates with a `threading.Lock`: `with lock: counter += 1`, or avoid sharing by using a `queue.Queue` or per-thread results. → *More:* Ch 16 §16.4 — Race conditions and locks

Q204. What is a deadlock? How do you avoid it? What is an `RLock`? A deadlock is when threads wait forever for locks held by each other (A holds lock 1 and waits for lock 2, B holds lock 2 and waits for lock 1). Avoid it by always acquiring locks in the same global order, holding locks briefly, using timeouts (`lock.acquire(timeout=1)`), and preferring queues to shared state. An `RLock` (re-entrant lock) can be acquired again by the thread that already holds it, which is useful when a locked method calls another locked method. → *More:* Ch 16 §16.4, §16.11

Q205. What synchronisation tools does `threading` offer? `Lock` and `RLock` (mutual exclusion), `Semaphore` / `BoundedSemaphore` (allow at most *n* holders, e.g. limit connections), `Event` (one thread signals, others wait), `Condition` (wait until a state changes), `Barrier` (wait until *n* threads arrive), plus thread-safe `queue.Queue`, `LifoQueue` and `PriorityQueue`. → *More:* Ch 16 §16.4–16.5

Q206. How do you implement producer–consumer in Python? Use `queue.Queue`, which is thread-safe and blocks: producers call `q.put(item)`, consumers loop on `item = q.get()` and call `q.task_done()`; `q.join()` waits until all items are processed. Use `Queue(maxsize=n)` for back-pressure (producers wait when consumers fall behind), and a sentinel such as `None` to tell consumers to stop. For processes use `multiprocessing.Queue`; for coroutines `asyncio.Queue`. → *More:* Ch 16 §16.5 — Producer–consumer with queue.Queue

Q207. What is `concurrent.futures`? What is the difference between `submit` / `as_completed` and `map`? A high-level API with `ThreadPoolExecutor` and `ProcessPoolExecutor` (and `InterpreterPoolExecutor` in 3.14) that share one interface. `ex.submit(f, x)` returns a `Future` (a placeholder for a result); `as_completed(futures)` yields them as they finish, so you can handle results and errors one by one. `ex.map(f, xs)` returns results in input order, and an exception surfaces when you reach that item. Use the executor as a context manager so it waits and shuts down cleanly. → *More:* Ch 16 §16.8 — Executors

Q208. What are the multiprocessing start methods? What changed in 3.14? Why is `if __name__ == "__main__"` required? `fork` copies the parent process (fast, but unsafe if the parent has threads); `spawn` starts a fresh interpreter and re-imports the main module (default on Windows and macOS); `forkserver` forks children from a clean single-threaded server process. Since 3.14, `forkserver` is the default on Linux and other Unix systems except macOS. Because `spawn` and `forkserver` re-import your script in each child, start-up code must sit under `if __name__ == "__main__":`, or every child would start children again. Arguments and results are pickled, so lambdas and local functions cannot be sent. → *More:* Ch 16 §16.7 — multiprocessing

Q209. How do processes share data? By message passing: `multiprocessing.Queue` and `Pipe` (objects are pickled). For large arrays, `multiprocessing.shared_memory.SharedMemory` (3.8+) gives a raw shared buffer that NumPy arrays can wrap without copying; `Value` and `Array` share simple C types; `Manager()` provides proxied lists and dicts (convenient but slow). Prefer designs where each worker gets its input and returns its result. → *More:* Ch 16 §16.7

Q210. What are subinterpreters (`concurrent.interpreters`, Python 3.14)? PEP 734 adds a standard-library module for running several isolated interpreters inside one process, each with its own GIL, modules and state. They give multi-core parallelism with lower overhead than processes; data is shared by explicit channels (`concurrent.interpreters` queues) rather than shared objects. `concurrent.futures.InterpreterPoolExecutor` provides the familiar executor interface. Many third-party C extensions do not support them yet.

Q211. ⋆ How does asyncio work? What is the event loop? asyncio runs many **coroutines** (functions defined with `async def`) on a single thread. The **event loop** keeps a list of ready tasks and runs one until it reaches an `await` on something not ready (a socket read, a timer); the coroutine then suspends and the loop runs another ready task, resuming the first when its I/O completes (the operating system notifies the loop). This is **cooperative multitasking**: tasks give up control only at `await`, so there are no surprise interruptions, but a task that never awaits blocks everyone. → *More:* Ch 20 Q43 · Ch 16 §16.9 — asyncio

Q212. What is the difference between a coroutine, a task and a thread? Calling an `async def` function returns a **coroutine** object that does nothing until awaited or scheduled. A **task** (`asyncio.create_task(coro)`) wraps a coroutine and schedules it to run concurrently on the event loop. A **thread** is an operating-system unit of execution, preemptively scheduled (the OS can switch it at any moment), with its own stack, costing much more memory than a task; asyncio can run hundreds of thousands of tasks on one thread. → *More:* Ch 16 §16.9

Q213. How do you run several coroutines concurrently? `gather` vs `TaskGroup` vs `wait`? `await asyncio.gather(c1, c2)` runs them concurrently and returns results in order; by default the first exception is propagated while the other tasks keep running (`return_exceptions=True` returns exceptions as values). `async with asyncio.TaskGroup() as tg: tg.create_task(...)` (3.11+) is the modern structured choice: if one task fails, the others are cancelled and the errors are raised as an `ExceptionGroup`. `asyncio.wait` gives low-level control (`FIRST_COMPLETED`). Timeouts: `async with asyncio.timeout(5):` (3.11) or `asyncio.wait_for`. → *More:* Ch 16 §16.9

Q214. ⋆ What happens if you call a blocking function like `time.sleep()` or `requests.get()` inside a coroutine? How do you fix it? It blocks the whole event loop: no other task runs until it returns, so the async program becomes sequential. Use async equivalents (`await asyncio.sleep()`, `httpx.AsyncClient`, `aiohttp`, async database drivers), or move blocking or CPU-heavy calls off the loop with `await asyncio.to_thread(func, *args)` (or `loop.run_in_executor` with a process pool for CPU work). → *More:* Ch 16 §16.9, §16.11 — Common pitfalls

Q215. How do you limit concurrency in asyncio, for example to at most 10 requests at a time? Use an `asyncio.Semaphore(10)`, and make each task `async with sem:` around the request; or start a fixed number of worker tasks reading from an `asyncio.Queue`. Both prevent overloading the remote server or running out of sockets.

```
sem = asyncio.Semaphore(10)
async def fetch(client, url):
    async with sem:
        return await client.get(url)
```

→ *More:* Ch 16 §16.9

Q216. What are daemon threads? A daemon thread (`Thread(target=f, daemon=True)`) does not keep the program alive: when only daemon threads remain, the interpreter exits and kills them abruptly, without running their `finally` blocks. Use them for background helpers (heartbeats, monitoring) that can be stopped at any point, never for work that must finish or flush data. Non-daemon threads should be `join()`ed. → *More:* Ch 16 §16.3 — Threads

Q217. What is the difference between `threading.local` and `contextvars`? `threading.local()` gives each thread its own copy of attributes. `contextvars.ContextVar` gives each *context* its own value; asyncio copies the context into every task, so it works correctly for coroutines on one thread (request IDs, current user, database session in async web frameworks), where thread-locals would be shared by all tasks. → *More:* Ch 16 §16.9

Q218. Are Python's built-in data structures thread-safe? Individual operations on built-ins such as `list.append`, `dict[key] = v` and `deque.append` are atomic in CPython (protected by the GIL, and by per-object locks in the free-threaded build), but this is an implementation detail and a *sequence* of operations (check-then-set, read-modify-write) is not atomic. Use locks for compound updates, `queue.Queue` for hand-offs between threads, and never rely on atomicity for correctness logic. → *More:* Ch 16 §16.3–16.4

---

12. Standard library, testing and debugging

Q219. Which standard-library modules do you use most? A good answer names a range with a use for each: `collections`, `itertools`, `functools` (data and functional tools); `pathlib`, `os`, `shutil`, `sys` (files and environment); `json`, `csv`, `sqlite3`, `tomllib` (data formats); `datetime` and `zoneinfo` (time); `re` (regex); `logging`, `argparse` (applications); `subprocess` (run programs); `dataclasses`, `enum`, `typing`; `concurrent.futures`, `asyncio`; `unittest`/`unittest.mock`; `random`, `secrets`, `hashlib`, `uuid`. → *More:* Ch 14 — A Tour of the Standard Library

Q220. What is the difference between naive and aware `datetime` objects? How do you handle time zones? A naive `datetime` has no time zone (`tzinfo is None`), so it is ambiguous; an aware one carries a time zone. Rules: store and compute in UTC (`datetime.now(timezone.utc)`), convert to local time only for display with `zoneinfo.ZoneInfo("Asia/Kolkata")` (3.9+), and never mix naive and aware values (comparisons raise

`TypeError`). `datetime.utcnow()` is deprecated because it returns a naive value. Use `timedelta` for arithmetic, `isoformat()`/`fromisoformat()` for strings. → *More:* Ch 14 §14.1 — datetime and zoneinfo

Q221. `random` vs `secrets`: which do you use for tokens and passwords? `random` is a fast pseudo-random generator (Mersenne Twister) that is predictable once its state is known; fine for simulations, games, sampling and shuffling (`random.seed` for reproducibility). For anything security-related (passwords, reset tokens, API keys) use `secrets`: `secrets.token_urlsafe(32)`, `secrets.choice`, `secrets.compare_digest` for constant-time comparison. → *More:* Ch 14 §14.7 — random vs secrets

Q222. Why use `logging` instead of `print`? How should you set it up? `logging` gives severity levels (DEBUG, INFO, WARNING, ERROR, CRITICAL), timestamps and source information, configurable destinations (console, files, external systems) and filtering, all without code changes; `print` gives none of that. Best practice: in each module `logger = logging.getLogger(__name__)`; configure handlers once in the application entry point (`logging.basicConfig` or `dictConfig`); use lazy formatting `logger.info("user %s", uid)`; use `logger.exception(...)` inside `except` to include the traceback; libraries should not configure logging. → *More:* Ch 14 §14.4 — logging · Ch 15 §15.12

Q223. How do you run an external command from Python safely? `subprocess.run(["git", "status"], capture_output=True, text=True, check=True, timeout=30)`. Pass the command as a list so no shell is involved; avoid `shell=True` with any user input, because it allows shell injection (`; rm -rf /`). `check=True` raises `CalledProcessError` on a non-zero exit code. `os.system` is obsolete. → *More:* Ch 14 §14.6 — os, sys, subprocess, shutil

Q224. What is the difference between `os`, `sys` and `shutil`? `os` talks to the operating system: environment variables (`os.environ`), processes, low-level file operations, `os.walk`. `sys` talks to the interpreter: `sys.argv`, `sys.path`, `sys.exit`, `sys.stdin/stdout`, `sys.version`, `sys.getrecursionlimit`. `shutil` provides high-level file operations: copying and moving trees, deleting directories, archives, `shutil.which`. → *More:* Ch 14 §14.6

Q225. How do you read command-line arguments? `sys.argv` is the raw list (the script name is `argv[0]`). For real tools use `argparse`: declare positional and optional arguments with types, defaults, choices and help text, and it parses, validates and generates `--help` automatically. Third-party alternatives: `click` and `typer` (which builds a CLI from type hints). → *More:* Ch 14 §14.5 — argparse

Q226. ⋆ What is the difference between `unittest` and `pytest`? Which do you prefer? `unittest` is in the standard library and uses classes inheriting `TestCase` with `self.assertEqual`-style methods and `setUp`/`tearDown`. `pytest` (third-party, the de-facto standard) uses plain functions and the `assert` statement (with detailed failure messages), fixtures for setup, `parametrize`, a rich plugin ecosystem, and it also runs `unittest` tests. Most teams prefer pytest for less boilerplate. → *More:* Ch 20 Q45 · Ch 15 §15.3–15.4

Q227. What are pytest fixtures? What are fixture scopes and `conftest.py`? A fixture is a function decorated with `@pytest.fixture` that provides a test with data or a resource; a test requests it by naming it as a parameter. Code after a `yield` in the fixture runs as teardown. Scope controls how often it is created: `function` (default), `class`, `module`, `package`, `session` (e.g. one database container per run). Fixtures in `conftest.py` are shared with all tests in that directory and below, without imports. Built-ins include `tmp_path`, `monkeypatch`, `capsys`. → *More:* Ch 15 §15.4 — pytest

Q228. What does `@pytest.mark.parametrize` do? It runs the same test function once for each set of arguments, reporting each as a separate test: `@pytest.mark.parametrize("a, b, expected", [(1, 2, 3), (0, 0, 0), (-1, 1, 0)])`. This replaces copy-pasted tests and loops inside a test (where the first failure hides the rest). Use `pytest.param(..., id="name")` for readable names. → *More:* Ch 15 §15.4

Q229. ⋆ What is mocking? Where should you patch? A mock is a fake object that replaces a real dependency (network, database, clock) so a test is fast and deterministic, and that records how it was called (`mock.assert_called_once_with(...)`). `unittest.mock.patch("target")` temporarily replaces a name. **Patch where the name is looked up**, not where it is defined: if `app/service.py` does `from requests import get`, patch `"app.service.get"`, not `"requests.get"`. Use `autospec=True` so the mock enforces the real signature. → *More:* Ch 20 Q46 · Ch 15 §15.5 — Test doubles and unittest.mock

Q230. What is the difference between `Mock` and `MagicMock`? What are stubs, fakes and spies? `Mock` creates attributes and methods on demand; `MagicMock` additionally supports dunder methods (`len(m)`, iteration, `with m:`); `AsyncMock` returns awaitables. Vocabulary: a *stub* returns canned answers, a *fake*

is a working lightweight implementation (an in-memory repository), a *spy* records calls to a real object, and a *mock* additionally verifies expected interactions. → *More:* Ch 15 §15.5

Q231. What is code coverage? Is 100 % coverage the goal? Coverage (`coverage.py`, `pytest --cov`) measures which lines (and branches, with `--branch`) ran during tests. It shows untested code, but high coverage does not prove tests check the right things: a test without assertions still "covers" lines. Aim for high coverage of important logic (commonly 80–90 %), check branches, and judge tests by whether they would catch real bugs. → *More:* Ch 15 §15.6 — Coverage

Q232. What is TDD? What is property-based testing? Test-driven development: write a failing test (red), write the minimum code to pass it (green), then clean up (refactor), in small cycles. Property-based testing (the Hypothesis library) states a rule that must always hold, such as "decoding an encoded value returns the original" or "sorting is idempotent", and the library generates hundreds of inputs, including edge cases, and shrinks any failure to a minimal example. → *More:* Ch 15 §15.8–15.9

Q233. How do you debug Python code? Read the traceback from the bottom (the exception and the line that raised it) upward through the call chain. Reproduce with a small test. Use `breakpoint()` (3.7+) to drop into `pdb` (`n` next, `s` step into, `c` continue, `p expr` print, `w` where, `u`/`d` move up/down the stack), or an IDE debugger; `python -m pdb -c continue script.py` enters the debugger on a crash, and 3.14 can attach to a running process with `python -m pdb -p PID`. Add logging for problems in production. → *More:* Ch 15 §15.10–15.11 — Tracebacks and pdb

Q234. What is a doctest? Examples written in a docstring in REPL form (`>>> add(2, 3)` followed by `5`) that the `doctest` module runs and compares with the shown output. They keep documentation examples correct, but are not a replacement for a real test suite. Run with `python -m doctest module.py -v` or `pytest --doctest-modules`. → *More:* Ch 15 §15.7 — doctest

---

13. "What does this print?" puzzles

Write down your answer before reading ours. All outputs were checked on CPython 3.14.8 (3.11 prints the same; only the traceback layout differs).

Q235. ⋆ What does this print?

```
a = [[]] * 3
a[0].append(1)
print(a)
```

`[[1], [1], [1]]`. `* 3` copies the *reference* three times, so all three slots point to one inner list. Create independent lists with `[[] for _ in range(3)]`. → *More:* Ch 5 §5.8 — the nested-list trap

Q236. ⋆ What does this print?

```
def add(x, items=[]):
    items.append(x)
    return items
print(add(1), add(2))
```

`[1, 2] [1, 2]`. The default list is created once and shared by both calls, and both calls return that same list, so by the time `print` runs it holds both values. Use `items=None`. → *More:* Ch 20 gotcha 1

Q237. ⋆ What does this print?

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])
```

`[2, 2, 2]`. Late binding: each lambda looks up `i` when called, after the loop ended with `i == 2`. Fix with `lambda i=i: i`. → *More:* Ch 20 gotcha 2

Q238. What does this print?

```
t = (1, [2])
try:
    t[1] += [3]
except TypeError as e:
    print(e)
print(t)
```

`'tuple' object does not support item assignment`, then `(1, [2, 3])`. `+=` first extends the list in place (succeeds), then tries to store the result back into the tuple slot (fails). The error is raised, yet the list has already changed. → *More:* Ch 5 §5.9

Q239. What does this print?

```
print(0.1 + 0.2 == 0.3, 0.1 * 3)
print(round(2.5), round(3.5))
```

`False 0.30000000000000004`, then `2 4`. Binary floating point cannot store 0.1 exactly; `round` uses round-half-to-even. → *More:* Ch 20 gotcha 10 · Ch 2 §2.5

Q240. What does this print?

```
print(-7 // 2, -7 % 2, int(-3.5), 2 ** 3 ** 2)
```

`-4 1 -3 512`. `//` floors toward minus infinity; `%` takes the sign of the divisor; `int()` truncates toward zero; `**` is right-associative (`2 ** 9`). → *More:* Ch 2 §2.8

Q241. ⋆ What does this print?

```
d = {1: "a", True: "b", 1.0: "c"}
print(d)
```

`{1: 'c'}`. `1 == True == 1.0` and they hash equally, so they are one key. The first key object (`1`) is kept and the value is overwritten twice. → *More:* Ch 6 §6.2

Q242. What does this print?

```
x = 10
def f():
    print(x)
    x = 5
f()
```

`UnboundLocalError: cannot access local variable 'x' where it is not associated with a value`. The assignment makes `x` local throughout `f`, so the `print` reads an unassigned local. → *More:* Ch 7 §7.3

Q243. What does this print?

```
g = (n for n in range(3))
print(list(g), list(g))
```

`[0, 1, 2] []`. A generator is an iterator and can be consumed only once. → *More:* Ch 11 §11.1

Q244. What does this print?

```
a = [1, 2, 3]
b = a
b += [4]
b = b + [5]
print(a, b)
```

`[1, 2, 3, 4] [1, 2, 3, 4, 5]`. `+=` mutates the shared list in place; `b + [5]` creates a new list and rebinds only `b`. → *More:* Ch 5 §5.4

Q245. ⋆ What does this print?

```
nums = [1, 2, 3, 4]
for n in nums:
    nums.remove(n)
print(nums)
```

`[2, 4]`. Removing an item shifts the rest left while the loop's internal index keeps advancing, so every other item is skipped. Iterate over a copy (`for n in nums[:]`) or build a new list with a comprehension. → *More:* Ch 20 gotcha 5

Q246. ⋆ What does this print?

```
class A:
    items = []
p, q = A(), A()
p.items.append(1)
print(q.items)

class C:
    n = 0
c = C()
c.n += 1
print(C.n, c.n)
```

`[1]`, then `0 1`. `items` is one class-level list shared by all instances. `c.n += 1` reads the class value 0, then *assigns* `c.n = 1` on the instance, shadowing the class attribute without changing it. → *More:* Ch 20 gotcha 8

Q247. What does this print?

```
class A:
    def hi(self): return "A"
class B(A):
    def hi(self): return "B" + super().hi()
class C(A):
    def hi(self): return "C" + super().hi()
class D(B, C):
    pass
print(D().hi(), [k.__name__ for k in D.__mro__])
```

`BCA ['D', 'B', 'C', 'A', 'object']`. `super()` in `B` follows the MRO of the instance's class (`D`), so it calls `C.hi`, not `A.hi`. → *More:* Ch 10 §10.5

Q248. What does this print?

```
def f():
    try:
        return "try"
    finally:
        print("finally runs")
print(f())
```

`finally runs`, then `try`. `finally` executes before the function actually returns, and here it does not override the value. (If `finally` itself had a `return`, its value would win, and 3.14 would emit a `SyntaxWarning`.) → *More:* Ch 9 §9.2

Q249. What does this print?

```
print(["a" "b", "c"])
print(type((1)), type((1,)))
```

`['ab', 'c']`, then `<class 'int'> <class 'tuple'>`. Adjacent string literals are concatenated at compile time (a missing comma becomes a silent bug). The comma, not the parentheses, makes a tuple. → *More:* Ch 5 §5.9

Q250. What does this print?

```
for i in range(3):
    pass
print(i)
print([x for x in range(3)], "x" in dir())
```

`2`, then `[0, 1, 2] False` (when run as a fresh script). A `for` loop variable survives after the loop in the enclosing scope, but a comprehension's variable is local to the comprehension and does not leak (in Python 3). → *More:* Ch 3 §3.6

Q251. What does this print?

```
print(1 < 2 < 3, 1 == 1.0 == True, (1 < 2) < 3)
print(all([]), any([]), bool("False"), bool([0]))
```

`True True True`, then `True False True True`. Chained comparisons combine with `and`; `True < 3` compares `1 < 3`. `all` of nothing is vacuously true; any non-empty string or list is truthy. → *More:* Ch 2 §2.6, §2.8

Q252. What does this print?

```
print(None or 0 or "x", 0 and 5, 3 and 5, [] or {})
```

`x 0 5 {}`. `or` returns the first truthy operand or the last one; `and` returns the first falsy operand or the last one. → *More:* Ch 2 §2.8

Q253. What does this print?

```
nums = [3, 1, 2]
result = nums.sort()
print(result, nums, sorted("bca"))
```

`None [1, 2, 3] ['a', 'b', 'c']`. In-place methods return `None`; `sorted` returns a new list and on a string gives a list of characters. → *More:* Ch 20 gotcha 13

Q254. What does this print?

```
s = "hello"
print(s[::-1], s[1:-1], s[-3:], s[10:], s[::2])
```

`olleh ell llo hlo` (the empty `s[10:]` produces two spaces in the output). Out-of-range slices return empty sequences instead of raising. → *More:* Ch 4 §4.4

Q255. What does this print?

```
import copy
m = [[1], [2]]
shallow = copy.copy(m)
shallow[0].append(9)
shallow.append([3])
print(m)
```

`[[1, 9], [2]]`. The shallow copy has its own outer list (so appending `[3]` does not affect `m`) but shares the inner lists. → *More:* Ch 5 §5.8

Q256. What does this print?

```
d = {"a": 1}
for k in d:
    d["b"] = 2
```

`RuntimeError: dictionary changed size during iteration`. Iterate over `list(d)` when you need to add or delete keys inside the loop. → *More:* Ch 6 §6.4

Q257. What does this print?

```
print([] == [], [] is [], "a" * 0, [0] * 3, [1, 2] * 2)
```

`True False [0, 0, 0] [1, 2, 1, 2]`. Two separate empty lists are equal but not identical; multiplying a sequence by 0 gives an empty one. → *More:* Ch 2 §2.2

Q258. What does this print?

```
def outer():
    n = 0
    def inc():
        nonlocal n
        n += 1
        return n
    return inc
c1, c2 = outer(), outer()
c1(); c1()
print(c1(), c2())
```

`3 1`. Each call to `outer` creates a new closure with its own `n`. → *More:* Ch 7 §7.4

Q259. What does this print?

```
print(f"{3.14159:.2f}|{42:05d}|{'hi':>5}|{1234567:,}|{0.25:.0%}")
```

`3.14|00042| hi|1,234,567|25%`. Format specs: precision, zero-padding with width, right alignment, thousands separator, percentage. → *More:* Ch 4 §4.6

Q260. What does this print?

```
x = [1, 2, 3]
y = x[:]
z = x
print(x == y, x is y, x is z)
print(list(zip([1, 2, 3], "ab")))
```

`True False True`, then `[(1, 'a'), (2, 'b')]`. A slice is a new list; `zip` stops at the shortest input (silently dropping `3`; use `strict=True` to catch that). → *More:* Ch 11 §11.7

---

14. Common coding tasks in Python rounds

These are the short tasks Python rounds use to see whether you write idiomatic code. Algorithm-heavy problems (two pointers, graphs, DP and so on) are covered pattern by pattern in Chapter 19; this section gives the Python-specific ones. Say the time complexity after each answer.

Q261. ⋆ Check whether a string is a palindrome (ignoring case and non-alphanumeric characters).

```
def is_palindrome(s: str) -> bool:
    cleaned = [c.lower() for c in s if c.isalnum()]
    return cleaned == cleaned[::-1]
```

O(n) time and space. Mention the O(1)-space two-pointer version (indices from both ends, skipping non-alphanumerics) as a follow-up. → *More:* Ch 19 §19.2 — Two pointers

Q262. ⋆ Check whether two strings are anagrams; group a list of words into anagrams.

```
from collections import Counter, defaultdict
def is_anagram(a: str, b: str) -> bool:
    return Counter(a) == Counter(b)

def group_anagrams(words: list[str]) -> list[list[str]]:
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)
    return list(groups.values())
```

`is_anagram` is O(n); grouping is O(n · k log k) for n words of length k (sorted letters as the key). → *More:* Ch 19 §19.3 — Hashing

Q263. ⋆ Count the frequency of each word in a text and return the top *k*.

```python
import re
from collections import Counter
def top_words(text: str, k: int) -> list[tuple[str, int]]:
    words = re.findall(r"[a-z']+", text.lower())
    return Counter(words).most_common(k)
```

O(n) to count, and `most_common(k)` uses a heap, O(m log k) for m distinct words. → *More:* Ch 6 §6.9

Q264. Find the first non-repeating character in a string.

```python
from collections import Counter
def first_unique(s: str) -> str | None:
    counts = Counter(s)
    return next((c for c in s if counts[c] == 1), None)
```

Two passes, O(n). `next(generator, default)` is the idiomatic "first match or default".

Q265. ⋆ Remove duplicates from a list while preserving order.

```python
def dedupe(items):
    return list(dict.fromkeys(items))      # dicts keep insertion order
```

O(n) for hashable items. `list(set(items))` loses the order. For unhashable items, keep a `seen` list or convert each item to a hashable key. → *More:* Ch 6 §6.1

Q266. Find the duplicates in a list.

```python
from collections import Counter
def duplicates(items):
    return sorted(x for x, n in Counter(items).items() if n > 1)
```

O(n log n) because of the sort (O(n) without it).

Q267. ⋆ Two Sum: return the indices of two numbers that add up to a target.

```python
def two_sum(nums: list[int], target: int) -> tuple[int, int] | None:
    seen: dict[int, int] = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return seen[target - n], i
        seen[n] = i
    return None
```

One pass, O(n) time and space, using a dict from value to index. → *More:* Ch 19 §19.3

Q268. ⋆ Flatten an arbitrarily nested list.

```python
def flatten(items):
    for x in items:
        if isinstance(x, list):
            yield from flatten(x)
        else:
            yield x
print(list(flatten([1, [2, [3, 4]], 5])))   # [1, 2, 3, 4, 5]
```

A recursive generator with `yield from`. For one level only: `[x for sub in items for x in sub]` or `itertools.chain.from_iterable`. Very deep nesting can hit the recursion limit; an explicit stack avoids it. → *More:* Ch 11 §11.3

Q269. Flatten a nested dictionary into dotted keys.

```python
def flatten_dict(d: dict, parent: str = "", sep: str = ".") -> dict:
    out = {}
    for k, v in d.items():
        key = f"{parent}{sep}{k}" if parent else k
        if isinstance(v, dict):
            out.update(flatten_dict(v, key, sep))
        else:
            out[key] = v
    return out
# {"a": {"b": 1, "c": {"d": 2}}} -> {"a.b": 1, "a.c.d": 2}
```

Q270. Deep-merge two nested dictionaries.

```python
def deep_merge(a: dict, b: dict) -> dict:
    out = dict(a)
    for k, v in b.items():
        if isinstance(v, dict) and isinstance(out.get(k), dict):
            out[k] = deep_merge(out[k], v)
        else:
            out[k] = v
    return out
```

Values from `b` win, nested dicts are merged recursively, and the inputs are not modified (useful for layered configuration).

Q271. Sort a dictionary by value; get the key with the maximum value.

```python
scores = {"ann": 82, "bob": 95, "cy": 82}
by_value = dict(sorted(scores.items(), key=lambda kv: kv[1], reverse=True))
best = max(scores, key=scores.get)          # 'bob'
```

O(n log n) to sort, O(n) for the max. `operator.itemgetter(1)` can replace the lambda. → *More:* Ch 5 §5.7

Q272. ⋆ Write a Fibonacci generator and a memoised Fibonacci function.

```python
from functools import cache
def fib_gen():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

@cache
def fib(n: int) -> int:
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

`itertools.islice(fib_gen(), 10)` takes the first ten. Plain recursion without the cache is $O(2^n)$; with it, O(n) (but recursion depth limits large n, so the iterative version is better there). → *More:* Ch 19 §19.10 — Dynamic programming

Q273. ⋆ Implement an LRU cache class.

```python
from collections import OrderedDict
class LRUCache:
    def __init__(self, capacity: int):
        self.capacity, self.data = capacity, OrderedDict()
    def get(self, key):
        if key not in self.data:
            return -1
        self.data.move_to_end(key)          # mark as most recently used
        return self.data[key]
    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)   # evict least recently used
```

Both operations are O(1). Interviewers may ask for the version with a dict plus a hand-written doubly linked list; the logic is the same. → *More:* Ch 6 §6.9 — the classic LRU cache

Q274. Write a retry decorator, or a memoise decorator, from scratch.

```python
import functools
def memoize(func):
    cache = {}
    @functools.wraps(func)
    def wrapper(*args):
        if args not in cache:
            cache[args] = func(*args)
        return cache[args]
    return wrapper
```

The cache lives in the closure; arguments must be hashable. For a retry decorator with arguments, see Ch 20 Q21; mention exponential back-off (`time.sleep(base * 2 ** attempt)`) and retrying only specific exceptions. → *More:* Ch 20 Q21 · Ch 12 §12.1

Q275. Split a list into chunks of size *n*.

```python
def chunks(items, n):
    return [items[i:i + n] for i in range(0, len(items), n)]
# Python 3.12+: list(itertools.batched(items, n)) gives tuples, and works on any iterable
```

→ *More:* Ch 11 §11.5

Q276. ⋆ Validate balanced brackets.

```python
def balanced(s: str) -> bool:
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
```

A list used as a stack; O(n). → *More:* Ch 19 §19.4 — Stacks and queues

Q277. Transpose a matrix and rotate it 90 degrees clockwise.

```python
m = [[1, 2, 3], [4, 5, 6]]
transposed = [list(row) for row in zip(*m)]        # [[1, 4], [2, 5], [3, 6]]
rotated = [list(row) for row in zip(*m[::-1])]     # [[4, 1], [5, 2], [6, 3]]
```

`zip(*m)` unpacks the rows as arguments, so it groups the i-th elements together. → *More:* Ch 11 §11.7

Q278. Find the missing number in a list containing 0..n with one number missing.

```python
def missing(nums: list[int]) -> int:
    n = len(nums)
    return n * (n + 1) // 2 - sum(nums)
```

O(n) time, O(1) extra space. An XOR of all indices and values also works and cannot overflow (not a concern for Python ints, but worth saying for other languages). → *More:* Ch 19 §19.12 — Bit tricks and math

Q279. Merge two sorted lists.

```python
def merge(a: list[int], b: list[int]) -> list[int]:
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out + a[i:] + b[j:]
```

O(n + m). Library shortcut: `list(heapq.merge(a, b))`, which also merges many sorted iterables lazily. → *More:* Ch 19 §19.17

Q280. Read a log file and report the 3 most frequent IP addresses.

```python
from collections import Counter
def top_ips(path: str, k: int = 3) -> list[tuple[str, int]]:
    with open(path, encoding="utf-8") as f:
        ips = (line.split(maxsplit=1)[0] for line in f if line.strip())
        return Counter(ips).most_common(k)
```

Streams the file with a generator, so memory stays constant for huge logs (only the counter grows). Mention `re` for stricter parsing and handling malformed lines. → *More:* Ch 9 §9.9

Q281. Implement your own `range`-like generator and your own version of `zip`.

```python
def my_range(start, stop=None, step=1):
    if stop is None:
        start, stop = 0, start
    while (step > 0 and start < stop) or (step < 0 and start > stop):
        yield start
        start += step

def my_zip(*iterables):
    iterators = [iter(it) for it in iterables]
    while iterators:
        row = []
        for it in iterators:
            try:
                row.append(next(it))
            except StopIteration:
                return            # stop at the shortest input
        yield tuple(row)
```

Talking point: writing `yield tuple(next(it) for it in iterators)` instead looks shorter but is wrong. The `StopIteration` escapes from inside a generator expression and is turned into `RuntimeError` (PEP 479, Q129). → *More:* Ch 11 §11.1–11.2

Q282. Write a thread-safe counter, then sum a list of URLs' sizes concurrently.

```python
import threading
from concurrent.futures import ThreadPoolExecutor
class SafeCounter:
    def __init__(self):
        self._value, self._lock = 0, threading.Lock()
    def increment(self):
        with self._lock:
            self._value += 1

def total_size(urls, fetch):              # fetch(url) -> bytes
    with ThreadPoolExecutor(max_workers=8) as ex:
        return sum(len(body) for body in ex.map(fetch, urls))
```

Threads suit network I/O; the lock makes the read-modify-write atomic. → *More:* Ch 16 §16.4, §16.8

Q283. Fetch many URLs concurrently with asyncio, at most 10 at a time.

```python
import asyncio
async def fetch_all(urls, fetch):         # fetch is an async function
    sem = asyncio.Semaphore(10)
    async def one(url):
        async with sem:
            return await fetch(url)
    async with asyncio.TaskGroup() as tg:
        tasks = [tg.create_task(one(u)) for u in urls]
    return [t.result() for t in tasks]
```

`TaskGroup` cancels the rest if one fails; the semaphore caps concurrency. Results come back in input order. → *More:* Ch 16 §16.9

Q284. Implement a stack and a queue class.

```python
from collections import deque
class Stack:
    def __init__(self): self._items = []
    def push(self, x): self._items.append(x)
    def pop(self): return self._items.pop()        # IndexError if empty
    def peek(self): return self._items[-1]
    def __len__(self): return len(self._items)

class Queue:
    def __init__(self): self._items = deque()
    def enqueue(self, x): self._items.append(x)
    def dequeue(self): return self._items.popleft()
```

All operations O(1). Using `list.pop(0)` for the queue would be O(n). → *More:* Ch 19 §19.4

Q285. Reverse the words in a sentence; capitalise each word.

```python
s = "  the quick   brown fox "
print(" ".join(reversed(s.split())))     # 'fox brown quick the'
print(" ".join(w.capitalize() for w in s.split()))
```

`split()` with no argument handles repeated spaces. `str.title()` also capitalises but mishandles apostrophes ("don'T"). → *More:* Ch 4 §4.7–4.8

Q286. Rotate a list by *k* positions.

```python
def rotate_right(xs: list, k: int) -> list:
    if not xs:
        return xs
    k %= len(xs)
    return xs[-k:] + xs[:-k]
```

O(n). In place: `deque.rotate(k)`, or the three-reversal trick for an array. → *More:* Ch 19 §19.2

Q287. Implement binary search.

```python
def binary_search(a: list[int], target: int) -> int:
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == target:
            return mid
        if a[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

O(log n) on a sorted list. In practice: `i = bisect.bisect_left(a, target)` then check `i < len(a) and a[i] == target`. → *More:* Ch 19 §19.6 — Binary search and bisect

---

15. NumPy and pandas basics

Asked in data-analyst, data-engineer, data-scientist and ML interviews, and often as a warm-up in general Python rounds at data-heavy companies.

Q288. Why is a NumPy array faster than a Python list? A NumPy `ndarray` stores numbers of one fixed type (its **dtype**, such as `int64` or `float64`) in one contiguous block of memory, without per-element Python objects. Operations on whole arrays run as compiled C loops (**vectorisation**), often 10–100× faster than Python loops, and use SIMD CPU instructions and less memory. Lists hold pointers to separate objects of any type, so every operation goes through the interpreter. → *More:* Ch 18 §18.1 — numpy arrays

Q289. What is broadcasting? NumPy's rule for combining arrays of different shapes without copying data. Shapes are compared from the right; two dimensions are compatible if they are equal or one of them is 1, and size-1 (or missing) dimensions are stretched virtually. So a `(3, 4)` array minus a `(4,)` array subtracts the row vector from every row, and `(3, 1)` with `(1, 4)` gives `(3, 4)`. Incompatible shapes raise `ValueError: operands could not be broadcast together`. → *More:* Ch 18 §18.2 — Broadcasting

Q290. What is the difference between a view and a copy in NumPy? Basic slicing (`a[1:3]`, `a[:, 0]`) and `reshape` (when possible) return **views**: new array objects that share the original data, so writing to them changes the original. Fancy indexing with lists or arrays (`a[[0, 2]]`) and boolean masks (`a[a > 0]`) return **copies**. Use `.copy()` when you need independence; `np.shares_memory(a, b)` checks. → *More:* Ch 18 §18.1

Q291. What does `axis` mean in NumPy and pandas? The axis is the dimension that an operation runs *along*, and which disappears from the result. For a 2-D array, `a.sum(axis=0)` sums down the rows and gives one value per column; `axis=1` sums across columns and gives one value per row. In pandas, `axis=0` (the default) means "per column" for aggregations, and `axis=1` means "per row" (and refers to columns in `drop`). → *More:* Ch 18 §18.1

Q292. How do you create arrays and random numbers the modern way? `np.array(list)`, `np.zeros((3, 4))`, `np.ones`, `np.arange(0, 10, 2)`, `np.linspace(0, 1, 5)`, `np.eye(3)`. For random numbers use a Generator: `rng = np.random.default_rng(seed=42)`, then `rng.random(5)`, `rng.integers(0, 10, size=3)`, `rng.normal(size=(2, 2))`. The old global `np.random.seed` / `np.random.rand` API is legacy. → *More:* Ch 18 §18.3 — Linear algebra and random numbers

Q293. What is the difference between a pandas Series and a DataFrame? How does pandas relate to NumPy? A `Series` is a one-dimensional labelled array (values plus an index). A `DataFrame` is a two-dimensional table: a set of Series (columns) sharing one row index, where each column can have its own dtype. pandas builds on NumPy (and optionally Apache Arrow) for storage and adds labels, automatic alignment on labels, missing-data handling, grouping, joins and time series. → *More:* Ch 18 §18.4 — Series and DataFrame

Q294. ⋆ What is the difference between `loc` and `iloc`? `loc` selects by **label** (index values and column names) and its slices *include* the end label: `df.loc["a":"c", ["price", "qty"]]`. `iloc` selects by integer **position** and its slices exclude the end like normal Python: `df.iloc[0:3, 0:2]`. Both accept boolean masks (`df.loc[df["qty"] > 5, "price"]`). `at` and `iat` are faster versions for a single cell. → *More:* Ch 18 §18.5 — Selecting data: loc, iloc and filters

Q295. How do you filter rows with multiple conditions? Combine boolean masks with `&` (and), `|` (or) and `~` (not), with **parentheses around each condition**, because these operators bind tighter than comparisons: `df[(df["age"] > 30) & (df["city"] == "Pune")]`. Python's `and` / `or` raise "The truth value of a Series is ambiguous". Alternatives: `df.query("age > 30 and city == 'Pune'")`, `df["city"].isin([...])`, `df["age"].between(20, 30)`. → *More:* Ch 18 §18.5

Q296. ⋆ How do you handle missing values in pandas? Detect with `df.isna()` and `df.isna().sum()` (count per column). Remove with `df.dropna()` (options `subset=`, `thresh=`). Fill with `df.fillna(value)` (a constant, a column mean or median, or a dict per column), or `ffill()` / `bfill()` for time series, or `interpolate()`. Choose based on why data is missing; sometimes add an indicator column. `NaN` is a float, so an integer column with missing values becomes float unless you use the nullable `Int64` dtype. → *More:* Ch 18 §18.6 — Cleaning: missing data and new columns

Q297. ⋆ Explain `groupby`. What is the difference between `agg`, `transform` and `apply` after a groupby? `groupby` follows split–apply–combine: split rows into groups by key, apply a function to each group,

combine the results. `agg` reduces each group to one row (`df.groupby("city")["sales"].agg(["sum", "mean"])`, or named aggregations). `transform` returns a result the same length as the original, aligned to each row, which is ideal for adding group statistics as a column (`df["share"] = df["sales"] / df.groupby("city")["sales"].transform("sum")`). `apply` runs arbitrary Python per group: flexible but slow. → *More:* Ch 18 §18.7 — groupby

Q298. ⋆ What is the difference between `merge`, `join` and `concat`? `pd.merge(left, right, on="key", how="inner")` is a SQL-style join on columns (`how` = `inner`, `left`, `right`, `outer`, `cross`). `df.join(other)` is a convenience that joins on the index. `pd.concat([a, b])` stacks frames vertically (`axis=0`) or side by side (`axis=1`), aligning on labels. Watch for duplicate keys multiplying rows; `validate="one_to_one"` or `"many_to_one"` checks your assumption, and `indicator=True` shows where each row came from. → *More:* Ch 18 §18.8 — Combining tables

Q299. What is the difference between `pivot`, `pivot_table` and `melt`? `pivot` reshapes long data to wide (index, columns, values) and fails if any index/column pair repeats. `pivot_table` does the same but aggregates duplicates (`aggfunc="sum"`), like a spreadsheet pivot, with optional totals (`margins=True`). `melt` goes the other way: wide to long, turning columns into rows. `stack`/`unstack` do similar reshaping using the index levels. → *More:* Ch 18 §18.9 — Reshaping

Q300. ⋆ Why is `apply` (or `iterrows`) slow, and what should you use instead? `apply(func, axis=1)` and `iterrows` call a Python function once per row, so they run at interpreter speed and build Python objects for each row. Use vectorised operations instead: column arithmetic (`df["a"] * df["b"]`), `np.where` or `np.select` for conditional logic, the `.str` and `.dt` accessors, `map` with a dict for lookups, and built-in group aggregations. If you must loop, `itertuples` is much faster than `iterrows`. → *More:* Ch 18 §18.11 — apply vs vectorised, and performance tips

Q301. What was `SettingWithCopyWarning`, and what changed in pandas 3.0 (Copy-on-Write)? In older pandas, chained indexing such as `df[df["x"] > 0]["y"] = 1` might modify a temporary copy instead of `df`, so pandas warned. In pandas 3.0, Copy-on-Write is always on: every indexing result *behaves* as a copy (memory is shared until one side is modified), so chained assignment never updates the original. `SettingWithCopyWarning` is gone; instead pandas emits a `ChainedAssignmentError` warning when it sees a chained assignment, which does nothing. Always assign in one step: `df.loc[df["x"] > 0, "y"] = 1`. pandas 3.0 also made `str` the default dtype for text columns. → *More:* Ch 18 §18.5 — Setting values and Copy-on-Write

Q302. How do you work with a CSV file that is too big for memory, or speed up pandas on large data? Read only what you need (`usecols=`, `dtype=` with smaller types, `category` for repeated strings, `parse_dates=`); process in pieces with `chunksize=` and aggregate per chunk; store data as Parquet (columnar, compressed, keeps dtypes) instead of CSV; and vectorise everything. Beyond one machine's memory, consider DuckDB, Polars (multi-threaded, lazy queries) or Dask/Spark. → *More:* Ch 18 §18.11

Q303. How do you find and remove duplicates, and count values, in pandas? `df.duplicated(subset=["email"])` returns a boolean mask, `df.drop_duplicates(subset=["email"], keep="last")` removes them. `df["city"].value_counts()` counts occurrences (`normalize=True` for proportions), and `df["city"].nunique()` counts distinct values. → *More:* Ch 18 §18.6

Q304. How do you work with dates in pandas? Parse with `pd.to_datetime(df["date"], format="%Y-%m-%d")`, then use the `.dt` accessor (`.dt.year`, `.dt.day_name()`, `.dt.to_period("M")`). With a `DatetimeIndex`, `resample("ME").sum()` aggregates by calendar period (pandas 3 uses `"ME"` for month end and `"h"` for hour), `rolling(7).mean()` gives moving averages, and `shift(1)` compares with the previous period. Use `tz_localize`/`tz_convert` for time zones. → *More:* Ch 18 §18.10 — Dates and times

Q305. How do you create a new column based on a condition? For two outcomes: `df["size"] = np.where(df["qty"] > 10, "big", "small")`. For several: `np.select([cond1, cond2], ["a", "b"], default="c")`. For binning numbers: `pd.cut(df["age"], bins=[0, 18, 65, 120], labels=["child", "adult", "senior"])`. For a mapping: `df["code"].map(lookup_dict)`. All are vectorised; avoid `apply` with an `if` per row. → *More:* Ch 18 §18.6

Sources of the question lists

Question wording and frequency were collected from these public lists and posts (titles only; all answers above are written for this book):

- **GitHub collections:** Devinterview-io, *python-interview-questions* ("100 Core Python Interview Questions"); baliyanvinay, *Python-Interview-Preparation* ("questions really asked in a Python interview", with coding questions); learning-zone, *python-basics* (Python interview questions by topic); softwaredeveloperhub01, *Python-Interview-Questions-and-Answers*; narayan09, *Python-Interview-Questions* (OOP); digitechit07, *Python-Interview-Questions-52*; pythonsheets.com, *Interview* notes.
- **Interview-experience posts and hiring guides:** LeetCode Discuss, *My 14 Coding Interview Experiences | 13 Rejections* (includes Python questions such as how list memory allocation works); Glassdoor question pages for "Python Developer", "Junior Python Developer", "Python Software Engineer" and "Senior Python Developer" (recurring: list vs tuple, decorators with parameters, dict comprehensions, GIL, memory management, class and abstract methods, generators, context managers, "what is the output" questions); InterviewKickstart, *Advanced Python Interview Questions*; GoLinuxCloud, *Python Developer Interview Questions*; Boundev, *Python interview questions hiring guide*; NovelVista and Naukri Python interview question lists.
- **Shared prep lists:** Scribd-hosted lists ("Top 25 Python Interview Questions", "25 Must-Know Python Interview Questions", "150+ Python Interview Questions") and Medium, *Top 100+ Python Coding Questions for Interview*, used to confirm which questions repeat across sources. (No public Notion page turned up in searches; these shared lists cover the same ground.)
- **LeetCode study lists (for Section 14):** LeetCode "Top Interview 150" and Blind 75 problems that Python rounds commonly use as warm-ups (Two Sum, Valid Anagram, Group Anagrams, Valid Parentheses, Missing Number, LRU Cache, Binary Search, Merge Two Sorted Lists, Rotate Array).
- **Data-stack questions (Section 15):** DataCamp, *Top Python pandas Interview Questions*; AskPython, *50 Essential Pandas Interview Questions*; GoLinuxCloud, *Pandas Interview Questions*; Educative, *pandas loc vs iloc*.
- **Version facts** were checked against the official *What's New in Python 3.12 / 3.13 / 3.14* pages, the Python 3.14.5 release notes (garbage collector revert), the PEPs named in the answers, and the pandas 3.0 release notes; outputs in Section 13 and the solutions in Section 14 were run on CPython 3.14.8, and the Section 15 snippets on numpy 2.4 and pandas 3.0.